NumPy Complete Practice Workbook

সম্পূর্ণ Solution ও বাংলা Explanation

288 Coding Problems | 25 Modules | 10 Research Mini-Projects

Basic থেকে Hard পর্যন্ত প্রতিটি problem-এর জন্য runnable Python + NumPy code, কোডের ধাপে ধাপে বাংলা ব্যাখ্যা এবং expected result দেওয়া হয়েছে।

বিশেষভাবে সাজানো হয়েছে Data Science, AI/ML এবং undergraduate research practice-এর জন্য।

Prepared from the uploaded 32-page NumPy coding workbook

How to use this Solution PDF

১. VS Code-এর Terminal খুলে NumPy install করো:

```
python -m pip install numpy
```

যদি ModuleNotFoundError আসে’ তাহলে VS Code-এ যে Python interpreter select করা আছে’ সেই interpreter-এর জন্য install করতে হবে. Windows-এ প্রয়োজনে চালাও:

```
py -m pip install numpy
```

২. প্রতিটি নতুন .py file বা notebook-এর প্রথমে পরের পৃষ্ঠার Shared Data Bank block একবার run করো. যেসব problem A, B, S, X, y, F, T বা Z ব্যবহার করে’ সেগুলো ওই data bank থেকে value নেবে.

৩. তারপর প্রয়োজনীয় problem-এর code আলাদা cell বা file-এ run করো. Code block-গুলো syntax-checked এবং shared data bank দিয়ে execution-tested করা হয়েছে.

৪. Expected result-এর সঙ্গে নিজের output মিলিয়ে দেখো. NumPy version, operating system বা floating-point rounding-এর কারণে কিছু decimal output সামান্য আলাদা হতে পারে.

৫. যেসব problem file save করে (যেমন .npy, .npz, .csv), সেগুলো current working directory-তে তৈরি হবে. Practice-এর জন্য আলাদা folder ব্যবহার করো; original research data overwrite কোরো না.

Important Research Rules

- Train/test split করার আগে পুরো dataset দিয়ে scaler বা imputer fit করবে না. Statistics শুধু training data থেকে শিখবে.

- Feature matrix X ও label vector y-কে সবসময় একই shuffled index দিয়ে reorder করবে.

- Random experiment-এ seed ব্যবহার করবে এবং প্রতিটি axis কী বোঝায় তা লিখে রাখবে.

- Missing value, infinity, zero variance ও feature unit আলাদা করে handle করবে.

Source note: Workbook-এর page 3-এ Shared Data Bank, page 4-এ 25-module curriculum map, page 5-29-এ 288 problems এবং page 30-31-এ 10টি research mini-project দেওয়া আছে. Workbook নিজেই CORE, EXTENDED ও RESEARCH exercise আলাদা করেছে; এই guide-এ সেই organization বজায় রাখা হয়েছে.

Shared Data Bank — Copy Once

Workbook-এর অনেক problem একই fictional dataset ব্যবহার করে· প্রতিটি practice file-এর শুরুতে এই block run করলে পরের solution-গুলো সরাসরি চালাতে পারবে·

```
import numpy as np

A = np.array([4, 8, 12, 16, 20])

B = np.array([
    [2, 4, 6],
    [8, 10, 12],
    [14, 16, 18]
])

C = np.array([
    [-4, 0, 3],
    [7, -2, 5]
])

# S: 4 students x 3 exams
S = np.array([
    [72, 80, 90],
    [65, 70, 75],
    [95, 88, 92],
    [60, 55, 62]
])

# X columns: message_length, url_count, suspicious_word_count
X = np.array([
    [120, 2, 4],
    [45, 0, 0],
    [200, 3, 7],
    [90, 1, 2],
    [160, 0, 5],
    [30, 0, 1]
])

# 1 = scam, 0 = non-scam
y = np.array([1, 0, 1, 0, 1, 0])

# F contains missing numeric measurements
F = np.array([
    [1.2, 0.5, 100],
    [2.4, np.nan, 200],
    [np.nan, 1.1, 150],
    [1.7, 0.7, np.nan],
    [2.0, 0.9, 180]
])

T = np.array([12, 15, 14, 20, 18, 22, 17])
Z = np.array([1, 2, 2, 3, 3, 3, 4, 5, 5])
```

Data Bank Explanation

A হলো 1D numeric array; B ও C matrix; S-এ row=student এবং column=exam; X-এ row=sample এবং column=message feature; y হলো binary label; F-এ NaN আছে; T numeric readings এবং Z repeated category values. X/y-এর scam label কেবল practice-এর জন্য—URL বা suspicious word count একা scam-এর প্রমাণ নয়·

Module Index

Module	Topic	Problems
01	NumPy setup and array mental model	N001–N010 (10)
02	Array creation methods	N011–N022 (12)
03	Shape, dimensions, dtype and casting	N023–N032 (10)
04	Indexing: reading and replacing elements	N033–N044 (12)
05	Slicing, steps and submatrices	N045–N056 (12)
06	Reshape, transpose, flatten and dimensions	N057–N068 (12)
07	Join, stack, split, repeat	N069–N078 (10)
08	Vectorized arithmetic and NumPy ufuncs	N079–N090 (12)
09	Aggregation, axis and keepdims	N091–N104 (14)
10	Broadcasting and shape compatibility	N105–N116 (12)
11	Comparisons and Boolean masks	N117–N128 (12)
12	Conditional transforms and clipping	N129–N138 (10)
13	Sort, find, rank and count unique values	N139–N150 (12)
14	Descriptive statistics for research	N151–N162 (12)
15	Random numbers and reproducible experiments	N163–N174 (12)
16	Views, copies and mutability	N175–N184 (10)
17	NaN, infinity and missing-data handling	N185–N196 (12)
18	Outliers, scaling and robust summaries	N197–N206 (10)
19	Vectors, matrices and linear algebra	N207–N218 (12)
20	Advanced indexing and 3D arrays	N219–N228 (10)
21	Saving and loading NumPy research data	N229–N238 (10)
22	Vectorization, efficiency and numerical precision	N239–N248 (10)
23	Research feature engineering with NumPy	N249–N262 (14)
24	ML math, predictions and evaluation from scratch	N263–N274 (12)
25	Debugging and output-prediction clinic	N275–N288 (14)

এরপর 10টি integrated research mini-project দেওয়া হয়েছে·

MODULE 01 | NumPy setup and array mental model

10টি problem | N001 থেকে N010

N001 — NumPy import ও version print

SOLUTION CODE

```
import numpy as np
print("NumPy version:", np.__version__)
```

EXPLANATION (BANGLA)

import numpy as np দিয়ে NumPy library-কে np নামে ব্যবহার করা যায়. __version__ হলো installed NumPy version-এর string property.

EXPECTED RESULT

Installed version, যেমন 2.x.x; version আপনার environment অনুযায়ী আলাদা হতে পারে.

N002 — Python list ও NumPy array তৈরি এবং type দেখা

SOLUTION CODE

```
import numpy as np
py_list = [3, 6, 9]
np_array = np.array([3, 6, 9])
print(py_list, type(py_list))
print(np_array, type(np_array))
```

EXPLANATION (BANGLA)

প্রথমটি built-in Python list; দ্বিতীয়টি NumPy ndarray. type() object-এর class দেখায়.

EXPECTED RESULT

list-এর type list; array-এর type numpy.ndarray.

N003 — List ও array-তে 3 দিয়ে multiplication তুলনা

SOLUTION CODE

```
import numpy as np
py_list = [2, 4, 6]
arr = np.array([2, 4, 6])
print(py_list * 3)
print(arr * 3)
```

EXPLANATION (BANGLA)

Python list * 3 করলে list-এর item তিনবার repeat হয়. NumPy array * 3 করলে প্রতিটি element-কে 3 দিয়ে গুণ করে.

EXPECTED RESULT

[2, 4, 6, 2, 4, 6, 2, 4, 6] এবং [6 12 18].

N004 — Loop ছাড়া array print

SOLUTION CODE

```
import numpy as np
arr = np.array([7, 14, 21])
print(arr)
```

EXPLANATION (BANGLA)

np.array() list-কে ndarray-তে রূপান্তর করে. print পুরো array একবারে দেখায়; explicit Python loop লাগে না.

EXPECTED RESULT

[7 14 21].

N005 — 0-D scalar ও 1-D one-element array

SOLUTION CODE

```
import numpy as np
scalar = np.array(5)
one_item = np.array([5])
print(scalar, scalar.ndim, scalar.shape)
print(one_item, one_item.ndim, one_item.shape)
```

EXPLANATION (BANGLA)

np.array(5)-এ কোনো axis নেই' তাই ndim=0 এবং shape=(). [5]-এ একটি axis আছে' তাই ndim=1 এবং shape=(1,).

EXPECTED RESULT

Scalar: ndim 0, shape (); one-item: ndim 1, shape (1,).

N006 — Data Bank A-এর metadata

SOLUTION CODE

```
print("A =", A)
print("len:", len(A))
print("shape:", A.shape)
print("size:", A.size)
print("ndim:", A.ndim)
print("dtype:", A.dtype)
```

EXPLANATION (BANGLA)

len() প্রথম axis-এর element count দেয়. shape হলো প্রতিটি axis-এর size; size মোট element; ndim axis সংখ্যা; dtype element-এর data type.

EXPECTED RESULT

A=[4 8 12 16 20], len=5, shape=(5,), size=5, ndim=1, dtype=int64 (সাধারণ 64-bit environment).

N007 — NumPy addition বনাম list addition

SOLUTION CODE

```
import numpy as np
a_list, b_list = [1, 2, 3], [4, 5, 6]
a, b = np.array(a_list), np.array(b_list)
print(a_list + b_list)
print(a + b)
```

EXPLANATION (BANGLA)

List-এর + concatenation করে. NumPy array-এর + element-wise যোগ করে: একই index-এর মানগুলো যোগ হয়.

EXPECTED RESULT

List: [1,2,3,4,5,6]; NumPy: [5 7 9].

N008 — Scalar broadcasting দিয়ে +10 ও *0

SOLUTION CODE

```
import numpy as np
arr = np.array([1, 2])
print(arr + 10)
print(arr * 0)
```

EXPLANATION (BANGLA)

একটি scalar array-এর প্রতিটি element-এর সঙ্গে operation করে. এই automatic expansion-কে broadcasting বলে.

EXPECTED RESULT

[11 12] এবং [0 0].

N009 — NumPy array চেনার function

SOLUTION CODE

```
import numpy as np
def is_numpy_array(obj):
    return isinstance(obj, np.ndarray)

print(is_numpy_array([1, 2]))
print(is_numpy_array((1, 2)))
print(is_numpy_array(np.array([1, 2])))
```

EXPLANATION (BANGLA)

isinstance(obj, np.ndarray) পরীক্ষা করে object-টি ndarray class-এর instance কি না. Function Boolean True/False ফেরত দেয়.

EXPECTED RESULT

False, False, True.

N010 — Scalar, vector ও 2D array-এর ndim/shape/size

SOLUTION CODE

```
import numpy as np
items = {
    "scalar": np.array(3),
    "vector": np.array([3]),
    "matrix": np.array([[3]])
}
for name, arr in items.items():
    print(name, "ndim=", arr.ndim,
          "shape=", arr.shape, "size=", arr.size)
```

EXPLANATION (BANGLA)

একই value 3 হলেও bracket-এর nesting dimension বদলায়. scalar 0-D, এক-element vector 1-D, আর 1x1 matrix 2-D.

EXPECTED RESULT

scalar: 0, (), 1; vector: 1, (1,), 1; matrix: 2, (1,1), 1.

MODULE 02 | Array creation methods

12টি problem | N011 থেকে N022

N011 — 1 থেকে 10 পর্যন্ত 1D array

SOLUTION CODE

```
arr = np.arange(1, 11)
print(arr)
```

EXPLANATION (BANGLA)

np.arange(start, stop) stop-কে বাদ দেয়· তাই 1 থেকে 10 পেতে stop=11 দিতে হয়·

EXPECTED RESULT

[1 2 3 4 5 6 7 8 9 10].

N012 — নির্দিষ্ট step দিয়ে [0,5,10,15,20]

SOLUTION CODE

```
arr = np.arange(0, 21, 5)
print(arr)
```

EXPLANATION (BANGLA)

তৃতীয় argument step=5; প্রতিবার 5 করে বাড়ে. stop=21 রাখায় 20 অন্তর্ভুক্ত হয়·

EXPECTED RESULT

[0 5 10 15 20].

N013 — 0 থেকে 1 inclusive পাঁচটি সমান দূরত্বের মান

SOLUTION CODE

```
arr = np.linspace(0, 1, 5)
print(arr)
```

EXPLANATION (BANGLA)

linspace(start, stop, num) নির্দিষ্ট সংখ্যক equally spaced value তৈরি করে এবং default-এ দুই endpoint-ই রাখে·

EXPECTED RESULT

[0.   0.25 0.5 0.75 1.].

N014 — 3x4 zeros array

SOLUTION CODE

```
arr = np.zeros((3, 4))
print(arr)
print(arr.shape)
```

EXPLANATION (BANGLA)

np.zeros((rows, columns)) নির্দিষ্ট shape-এর সব element শূন্য করে. Default dtype float.

EXPECTED RESULT

৩ row, ৪ column; shape=(3,4).

N015 — 2x5 integer ones array

SOLUTION CODE

```
arr = np.ones((2, 5), dtype=int)
print(arr)
print(arr.dtype)
```

EXPLANATION (BANGLA)

np.ones shape অনুযায়ী সব element 1 করে. dtype=int দিলে integer storage হয়·

EXPECTED RESULT

২x৫ matrix-এ সব 1; dtype integer.

N016 — 4x3 array-তে -1 বসানো

SOLUTION CODE

```
arr = np.full((4, 3), -1)
print(arr)
```

EXPLANATION (BANGLA)

np.full(shape, fill_value) প্রতিটি position-এ একই value বসায়; nested list লিখতে হয় না·

EXPECTED RESULT

৪ row, ৩ column-এর সব value -1.

N017 — 4x4 identity matrix ও nonzero count

SOLUTION CODE

```
eye = np.eye(4, dtype=int)
print(eye)
print(np.count_nonzero(eye))
```

EXPLANATION (BANGLA)

Identity matrix-এর main diagonal-এ 1 এবং অন্য জায়গায় 0 থাকে. count_nonzero() শূন্য নয় এমন element গোনে.

EXPECTED RESULT

Diagonal-এ চারটি 1; nonzero count=4.

N018 — B-এর shape ও dtype রেখে zeros তৈরি

SOLUTION CODE

```
zeros_like_B = np.zeros_like(B)
print(zeros_like_B)
print(zeros_like_B.shape, zeros_like_B.dtype)
```

EXPLANATION (BANGLA)

zeros_like() input array-এর shape ও dtype অনুসরণ করে, কিন্তু সব value 0 করে.

EXPECTED RESULT

shape=(3,3), B-এর মতো integer dtype.

N019 — প্রথম ১২টি positive even integer

SOLUTION CODE

```
evens = np.arange(2, 25, 2)
print(evens)
print(evens[0], evens[-1], len(evens))
```

EXPLANATION (BANGLA)

2 থেকে শুরু, step=2. 24 পর্যন্ত পেতে stop=25; শেষ value 24.

EXPECTED RESULT

[2,4,6,8,10,12,14,16,18,20,22,24]; first=2, last=24, length=12.

N020 — -2 থেকে 2 inclusive ১১টি মান ও spacing

SOLUTION CODE

```
values = np.linspace(-2, 2, 11)
print(values)
print(np.diff(values))
```

EXPLANATION (BANGLA)

linspace ১১টি মান তৈরি করে. np.diff পাশাপাশি element-এর পার্থক্য বের করে; এখানে spacing প্রায় 0.4.

EXPECTED RESULT

[-2,-1.6,...,1.6,2]; প্রতিটি consecutive distance 0.4.

N021 — Diagonal-এ [2,4,6,8] বসানো

SOLUTION CODE

```
diag = np.diag([2, 4, 6, 8])
print(diag)
```

EXPLANATION (BANGLA)

np.diag(vector) vector-এর value-গুলোকে main diagonal-এ বসায় এবং বাকি ঘর 0 রাখে.

EXPECTED RESULT

4x4 diagonal matrix.

N022 — arange ও linspace তুলনা

SOLUTION CODE

```
a = np.arange(0, 1, 0.1)
b = np.linspace(0, 1, 11)
print(a, a.size, a[-1])
print(b, b.size, b[-1])
```

EXPLANATION (BANGLA)

arange step ধরে এগোয় এবং stop=1 বাদ দেয়; floating-point step-এর কারণে rounding nuance থাকতে পারে. linspace নির্দিষ্ট ১১টি point দেয় এবং endpoint 1 রাখে.

EXPECTED RESULT

arange সাধারণত 10টি value, শেষ প্রায় 0.9; linspace 11টি value, শেষ 1.0.

MODULE 03 | Shape, dimensions, dtype and casting

10টি problem | N023 থেকে N032

N023 — B-এর metadata ও rows/columns

SOLUTION CODE

```
print(B.ndim, B.shape, B.size, B.dtype)
rows, columns = B.shape
print("rows:", rows, "columns:", columns)
```

EXPLANATION (BANGLA)

2D array-এর shape=(rows, columns). ndim=2; size=rows*columns.

EXPECTED RESULT

B: ndim=2, shape=(3,3), size=9; 3 rows ও 3 columns.

N024 — Float array তৈরি

SOLUTION CODE

```
arr = np.array([2.5, 3.0, 4.5], dtype=float)
print(arr, arr.dtype)
```

EXPLANATION (BANGLA)

dtype=float দিলে decimal value-গুলো floating-point format-এ থাকে।

EXPECTED RESULT

[2.5 3. 4.5], float dtype.

N025 — Boolean array

SOLUTION CODE

```
arr = np.array([True, False, True])
print(arr, arr.shape, arr.dtype)
```

EXPLANATION (BANGLA)

True/False হলো Boolean values. NumPy এদের bool dtype-এ রাখে।

EXPECTED RESULT

shape=(3,), dtype=bool.

N026 — Integer থেকে float-এ astype

SOLUTION CODE

```
arr = np.array([1, 2, 3])
float_arr = arr.astype(float)
print(float_arr, float_arr.dtype)
```

EXPLANATION (BANGLA)

astype() নতুন array তৈরি করে target dtype-এ convert করে. Original array বদলায় না।

EXPECTED RESULT

[1. 2. 3.], float dtype.

N027 — Float থেকে integer conversion

SOLUTION CODE

```
arr = np.array([1.2, 2.9, -3.7])
int_arr = arr.astype(int)
print(int_arr)
```

EXPLANATION (BANGLA)

float থেকে int conversion fractional অংশ কেটে দেয় (truncate toward zero); round করে না।

EXPECTED RESULT

[1 2 -3].

N028 — String সংখ্যাকে integer করে sum

SOLUTION CODE

```
text_values = np.array(["5", "10", "15"])
numbers = text_values.astype(int)
print(numbers)
print(numbers.sum())
```

EXPLANATION (BANGLA)

প্রথমে numeric string-গুলো int-এ cast হয়, এরপর sum() যোগ করে।

EXPECTED RESULT

[5 10 15], sum=30.

N029 — Mixed type promotion

SOLUTION CODE

```
arr = np.array([1, 2.5, True])
print(arr)
print(arr.dtype)
```

EXPLANATION (BANGLA)

একটি ndarray সাধারণত একই dtype রাখে' এখানে float-এর কারণে integer ও Boolean value-ও float-এ promote হয়; True হয় 1.0.

EXPECTED RESULT

[1. 2.5 1.], float dtype.

N030 — float32 বনাম float64 memory

SOLUTION CODE

```
a = np.ones(1000, dtype=np.float32)
b = np.ones(1000, dtype=np.float64)
print(a.nbytes, b.nbytes)
```

EXPLANATION (BANGLA)

nbytes array-এর element data-এর মোট byte দেখায়. float32 প্রতি element 4 byte; float64 8 byte.

EXPECTED RESULT

4000 bytes বনাম 8000 bytes.

N031 — squeeze দিয়ে singleton axis বাদ

SOLUTION CODE

```
arr = np.arange(5).reshape(1, 5)
squeezed = np.squeeze(arr)
print(arr.shape, squeezed.shape)
```

EXPLANATION (BANGLA)

squeeze() যেসব axis-এর length 1, শুধু সেগুলো বাদ দেয়' এখানে (1,5) থেকে (5,) হয়'

EXPECTED RESULT

আগে shape=(1,5), পরে shape=(5,).

N032 — int64 বনাম uint8 storage

SOLUTION CODE

```
labels64 = np.arange(1000, dtype=np.int64) % 6
labels8 = labels64.astype(np.uint8)
print(labels64.nbytes, labels8.nbytes)
print(np.array_equal(labels64, labels8))
```

EXPLANATION (BANGLA)

0-5 label uint8-এ নিরাপদে রাখা যায়; uint8-এর range 0-255. কিন্তু বড়/negative সংখ্যা থাকলে overflow বা ভুল conversion হতে পারে'

EXPECTED RESULT

int64=8000 bytes, uint8=1000 bytes; values একই'

MODULE 04 | Indexing: reading and replacing elements

12টি problem | N033 থেকে N044

N033 — A-এর first, third ও last element

SOLUTION CODE

```
print(A[0], A[2], A[-1])
```

EXPLANATION (BANGLA)

Index 0 থেকে শুরু হয়. A[2] তৃতীয় element; -1 শেষ element বোঝায়.

EXPECTED RESULT

4, 12, 20.

N034 — Negative index দিয়ে শেষ element

SOLUTION CODE

```
print(A[-1])
```

EXPLANATION (BANGLA)

Negative indexing ডান দিক থেকে গণনা করে; -1 মানে শেষ element.

EXPECTED RESULT

20.

N035 — B-এর চারপাশের তিনটি নির্দিষ্ট element

SOLUTION CODE

```
print(B[0, 0])
print(B[1, 1])
print(B[2, 2])
```

EXPLANATION (BANGLA)

2D indexing-এর নিয়ম arr[row, column]. top-left=(0,0), center=(1,1), bottom-right=(2,2).

EXPECTED RESULT

2, 10, 18.

N036 — B-এর দ্বিতীয় row

SOLUTION CODE

```
print(B[1])
```

EXPLANATION (BANGLA)

একটি index দিলে 2D array-এর নির্দিষ্ট row পাওয়া যায়. দ্বিতীয় row-এর index 1.

EXPECTED RESULT

[8 10 12].

N037 — B-এর তৃতীয় column

SOLUTION CODE

```
print(B[:, 2])
```

EXPLANATION (BANGLA)

প্রথম colon সব row নেয়; column index 2 তৃতীয় column নেয়.

EXPECTED RESULT

[6 12 18].

N038 — A-এর copy-তে first element replace

SOLUTION CODE

```
changed = A.copy()
changed[0] = 99
print(changed)
print(A)
```

EXPLANATION (BANGLA)

copy() independent array তৈরি করে. তাই changed বদলালেও original A অপরিবর্তিত থাকে.

EXPECTED RESULT

changed=[99,8,12,16,20]; A=[4,8,12,16,20].

N039 — B-এর center element -10 করা

SOLUTION CODE

```
changed = B.copy()
changed[1, 1] = -10
print(changed)
```

EXPLANATION (BANGLA)

copy নেওয়ার পর row 1, column 1 position-এ assignment করা হয়েছে; অন্য value অক্ষত থাকে।

EXPECTED RESULT

Center value 10-এর জায়গায় -10.

N040 — তৃতীয় শিক্ষার্থীর তৃতীয় exam score 100

SOLUTION CODE

```
scores = S.copy()
scores[2, 2] = 100
print(scores)
print("Original:", S[2, 2])
```

EXPLANATION (BANGLA)

তৃতীয় student-এর row index 2; তৃতীয় exam-এর column index 2. copy-তে বদলানো হয়েছে।

EXPECTED RESULT

নতুন matrix-এ S[2,2]=100; original value 92 থাকে।

N041 — X থেকে sample feature পড়া

SOLUTION CODE

```
print("4th message length:", X[3, 0])
print("6th message URL count:", X[5, 1])
```

EXPLANATION (BANGLA)

চতুর্থ sample-এর row index 3; message length column 0. ষষ্ঠ sample row 5; URL count column 1.

EXPECTED RESULT

90 এবং 0.

N042 — B-এর last row ও last column

SOLUTION CODE

```
print(B[-1, :])
print(B[:, -1])
```

EXPLANATION (BANGLA)

-1 row index শেষ row; -1 column index শেষ column. colon অন্য axis-এর সব element নেয়।

EXPECTED RESULT

Last row [14 16 18]; last column [6 12 18].

N043 — Index validation-সহ get_item function

SOLUTION CODE

```
def get_item(matrix, row, col):
    if not (0 <= row < matrix.shape[0]):
        raise IndexError("row index out of range")
    if not (0 <= col < matrix.shape[1]):
        raise IndexError("column index out of range")
    return matrix[row, col]

print(get_item(B, 1, 2))
try:
    print(get_item(B, 5, 0))
except IndexError as error:
    print(error)
```

EXPLANATION (BANGLA)

Function প্রথমে row ও column সীমার মধ্যে আছে কি না পরীক্ষা করে। ভুল index হলে পরিষ্কার IndexError দেয়; valid হলে element ফেরত দেয়।

EXPECTED RESULT

Valid call-এ 12; invalid call-এ row index out of range.

N044 — চার corner clockwise order-এ ফেরত

SOLUTION CODE

```
def corners_clockwise(matrix):
    if matrix.ndim != 2 or min(matrix.shape) < 2:
        raise ValueError("Need a 2D array with at least 2 rows and 2 columns")
    return np.array([
        matrix[0, 0], matrix[0, -1],
        matrix[-1, -1], matrix[-1, 0]
    ])

print(corners_clockwise(B))
```

EXPLANATION (BANGLA)

চারটি corner হলো top-left, top-right, bottom-right, bottom-left. clockwise order-এ নতুন array বানানো হয়েছে.

EXPECTED RESULT

B-এর ক্ষেত্রে [2, 6, 18, 14].

MODULE 05 | Slicing, steps and submatrices

12টি problem | N045 থেকে N056

N045 — A-এর প্রথম তিনটি value

SOLUTION CODE

```
print(A[:3])
```

EXPLANATION (BANGLA)

slice-এর stop index অন্তর্ভুক্ত হয় না। তাই :3 মানে index 0,1,2.

EXPECTED RESULT

[4 8 12].

N046 — A-এর শেষ দুইটি value

SOLUTION CODE

```
print(A[-2:])
```

EXPLANATION (BANGLA)

-2 থেকে শেষ পর্যন্ত slice করলে শেষ দুইটি element পাওয়া যায়।

EXPECTED RESULT

[16 20].

N047 — A reverse করা

SOLUTION CODE

```
print(A[::-1])
```

EXPLANATION (BANGLA)

slice-এর step -1 হলে array উল্টো দিক থেকে পড়া হয়।

EXPECTED RESULT

[20 16 12 8 4].

N048 — arange(20) থেকে odd index-এর প্রতি দ্বিতীয় item

SOLUTION CODE

```
arr = np.arange(20)
print(arr[1::2])
```

EXPLANATION (BANGLA)

start index 1, stop omitted (শেষ পর্যন্ত), step 2. ফলে index 1,3,5,... নেওয়া হয়।

EXPECTED RESULT

[1 3 5 7 9 11 13 15 17 19].

N049 — Index 5 থেকে 12 inclusive

SOLUTION CODE

```
arr = np.arange(30)
print(arr[5:13])
```

EXPLANATION (BANGLA)

Python slice-এ stop বাদ যায়; তাই index 12 রাখতে stop=13 দিতে হয়।

EXPECTED RESULT

[5 6 7 8 9 10 11 12].

N050 — B-এর top-left 2x2 block

SOLUTION CODE

```
print(B[:2, :2])
```

EXPLANATION (BANGLA)

প্রথম slice প্রথম দুই row নেয়; দ্বিতীয় slice প্রথম দুই column নেয়।

EXPECTED RESULT

[[2,4],[8,10]].

N051 — B-এর সব row, column 0 ও 2

SOLUTION CODE

```
print(B[:, ::2])
```

EXPLANATION (BANGLA)

Column slice ::2 index 0 থেকে প্রতি 2 ঘর পর নেয়: 0 এবং 2.

EXPECTED RESULT

[[2,6],[8,12],[14,18]].

N052 — B-এর row ও column order reverse

SOLUTION CODE

```
print("Rows reversed:\n", B[::-1, :])
print("Columns reversed:\n", B[:, ::-1])
```

EXPLANATION (BANGLA)

প্রথম expression row axis উল্টায়; দ্বিতীয় expression column axis উল্টায়.

EXPECTED RESULT

একটিতে row order উল্টো, অন্যটিতে প্রতিটি row-এর column order উল্টো.

N053 — Student 2-4-এর Exam 1 ও 2

SOLUTION CODE

```
print(S[1:4, 0:2])
```

EXPLANATION (BANGLA)

Workbook-এ student numbering 1-based বলা হয়েছে. Python index 0-based, তাই student 2-4 = row slice 1:4; exam 1-2 = column 0:2.

EXPECTED RESULT

shape=(3,2); [[65,70],[95,88],[60,55]].

N054 — 5x5 matrix-এর center 3x3 block

SOLUTION CODE

```
grid = np.arange(1, 26).reshape(5, 5)
print(grid[1:4, 1:4])
```

EXPLANATION (BANGLA)

1 থেকে 25 পর্যন্ত value 5x5-এ reshape. মাঝের row ও column index 1,2,3 নেওয়া হয়েছে.

EXPECTED RESULT

[[7,8,9],[12,13,14],[17,18,19]].

N055 — Odd-length 1D array-এর middle three

SOLUTION CODE

```
def middle_three(arr):
    arr = np.asarray(arr)
    if arr.ndim != 1 or arr.size < 3 or arr.size % 2 == 0:
        raise ValueError("Need an odd-length 1D array of at least 3 items")
    mid = arr.size // 2
    return arr[mid - 1:mid + 2]

print(middle_three(np.arange(1, 8)))
```

EXPLANATION (BANGLA)

Length odd ও অন্তত 3 কি না validate করা হয়. mid=n//2; mid-এর আগের, মাঝের ও পরের element slice করা হয়.

EXPECTED RESULT

Input [1,2,3,4,5,6,7] হলে [3,4,5].

N056 — প্রতি দ্বিতীয় row/column-এর view পরীক্ষা

SOLUTION CODE

```
matrix = np.arange(36).reshape(6, 6)
view = matrix[::2, ::2]
print(view.shape, np.shares_memory(matrix, view))
view[0, 0] = 999
print(matrix[0, 0])
```

EXPLANATION (BANGLA)

Basic slicing সাধারণত view তৈরি করে; view-এর element বদলালে original matrix-ও বদলায়. shares_memory() memory ভাগ হচ্ছে কি না পরীক্ষা করে.

EXPECTED RESULT

view shape=(3,3), shares_memory=True; original matrix[0,0] হয় 999.

MODULE 06 | Reshape, transpose, flatten and dimensions

12টি problem | N057 থেকে N068

N057 — 12 value-কে 3x4 matrix করা

SOLUTION CODE

```
arr = np.arange(12).reshape(3, 4)
print(arr)
print(arr.shape)
```

EXPLANATION (BANGLA)

reshape(3,4)-এ 3*4=12 element প্রয়োজন. Element order row-wise বজায় থাকে'

EXPECTED RESULT

shape=(3,4), value 0 থেকে 11.

N058 — 12 value-কে 2x2x3 array করা

SOLUTION CODE

```
arr = np.arange(12).reshape(2, 2, 3)
print(arr.shape)
print(arr)
```

EXPLANATION (BANGLA)

2*2*3=12, তাই 12 value এই 3D shape-এ সাজানো যায়'

EXPECTED RESULT

shape=(2,2,3).

N059 — -1 দিয়ে column সংখ্যা infer

SOLUTION CODE

```
arr = np.arange(15).reshape(3, -1)
print(arr.shape)
```

EXPLANATION (BANGLA)

reshape-এ একটি dimension -1 দিলে NumPy মোট element count থেকে সেটি হিসাব করে. 15/3=5.

EXPECTED RESULT

shape=(3,5).

N060 — B transpose করা

SOLUTION CODE

```
transposed = B.T
print(transposed)
print("before:", B.shape, "after:", transposed.shape)
```

EXPLANATION (BANGLA)

.T rows ও columns অদলবদল করে. B square 3x3 হওয়ায় shape একই থাকে' কিন্তু values-এর অবস্থান বদলায়'

EXPECTED RESULT

shape আগে ও পরে (3,3); matrix transpose হয়েছে'

N061 — B এক dimension-এ flatten

SOLUTION CODE

```
flat = B.flatten()
print(flat)
print(flat.shape)
```

EXPLANATION (BANGLA)

flatten() multi-dimensional array-কে 1D array বানায় এবং independent copy ফেরত দেয়'

EXPECTED RESULT

[2 4 6 8 10 12 14 16 18], shape=(9,).

N062 — 1D vector-কে column vector করা

SOLUTION CODE

```
v = np.array([10, 20, 30])
column = v.reshape(3, 1)
print(column, column.shape)
```

EXPLANATION (BANGLA)

3টি row ও 1টি column দিয়ে column vector তৈরি করা হয়েছে'

EXPECTED RESULT

shape=(3,1).

N063 — 1D vector-কে row vector করা

SOLUTION CODE

```
v = np.array([10, 20, 30])
row = v.reshape(1, 3)
print(row, row.shape)
```

EXPLANATION (BANGLA)

1টি row ও 3টি column দিয়ে row vector তৈরি করা হয়েছে।

EXPECTED RESULT

shape=(1,3); column vector-এর shape থেকে আলাদা।

N064 — A-তে leading axis যোগ

SOLUTION CODE

```
expanded = np.expand_dims(A, axis=0)
print(expanded.shape)
```

EXPLANATION (BANGLA)

axis=0 position-এ নতুন dimension ঢোকানো হয়. 1D shape (5,) থেকে (1,5).

EXPECTED RESULT

shape=(1,5).

N065 — 3D array-এর first ও last axis swap

SOLUTION CODE

```
arr = np.zeros((2, 3, 4))
swapped = np.swapaxes(arr, 0, 2)
print(swapped.shape)
```

EXPLANATION (BANGLA)

swapaxes(arr,0,2) axis 0 ও axis 2-এর position বদলায়; মাঝের axis অপরিবর্তিত থাকে।

EXPECTED RESULT

আগে (2,3,4), পরে (4,3,2).

N066 — ravel ও flatten copy/view তুলনা

SOLUTION CODE

```
base = B.copy()
r = base.ravel()
f = base.flatten()
r[0] = -100
f[1] = -200
print("base:\n", base)
print("ravel:", r)
print("flatten:", f)
```

EXPLANATION (BANGLA)

Contiguous array-তে ravel সাধারণত view দিতে পারে; flatten সবসময় copy দেয়। তাই r বদলালে base বদলাতে পারে, f বদলালে base বদলায় না।

EXPECTED RESULT

base-এর প্রথম value -100 হয়; flatten-এর পরিবর্তন base-এ যায় না।

N067 — ভুল reshape error ধরা

SOLUTION CODE

```
arr = np.arange(10)
try:
    arr.reshape(3, 4)
except ValueError as error:
    print("Cannot reshape:", error)
```

EXPLANATION (BANGLA)

reshape-এ element count একই থাকতে হবে. 10টি element দিয়ে 3x4=12 ঘর পূরণ করা যায় না।

EXPECTED RESULT

ValueError ধরা পড়ে; reshape ব্যর্থ হয়।

N068 — যেকোনো n x m matrix vector করে আবার reconstruct

SOLUTION CODE

```
matrix = np.arange(12).reshape(3, 4)
n, m = matrix.shape
vector = matrix.reshape(-1)
restored = vector.reshape(n, m)
print(np.array_equal(matrix, restored))
```

EXPLANATION (BANGLA)

প্রথমে original shape সংরক্ষণ করা হয়। তারপর 1D করা এবং আগের shape-এ ফেরত আনা হয়।

EXPECTED RESULT

True; element order ও values অক্ষত।

MODULE 07 | Join, stack, split, repeat

10টি problem | N069 থেকে N078

N069 — দুইটি 1D array concatenate

SOLUTION CODE

```
a = np.array([1, 2])
b = np.array([3, 4, 5])
print(np.concatenate((a, b)))
```

EXPLANATION (BANGLA)

concatenate() একই axis বরাবর array জোড়া লাগায়. 1D array-তে একটিই axis থাকে.

EXPECTED RESULT

[1 2 3 4 5].

N070 — দুইটি 1D array vertical stack

SOLUTION CODE

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print(np.vstack((a, b)))
```

EXPLANATION (BANGLA)

vstack 1D array-গুলোকে row হিসেবে stack করে 2D matrix বানায়.

EXPECTED RESULT

shape=(2,3); দুইটি row.

N071 — দুইটি 2x2 array horizontal join

SOLUTION CODE

```
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])
joined = np.hstack((a, b))
print(joined, joined.shape)
```

EXPLANATION (BANGLA)

hstack column direction-এ জোড়া দেয়. Row count একই থাকতে হবে.

EXPECTED RESULT

shape=(2,4).

N072 — 12 value-কে তিনটি সমান অংশে split

SOLUTION CODE

```
parts = np.split(np.arange(12), 3)
for part in parts:
    print(part)
```

EXPLANATION (BANGLA)

np.split(array,3) সমান 3 ভাগ করে; array length 12 হওয়ায় প্রতিটি অংশে 4টি value.

EXPECTED RESULT

[0 1 2 3], [4 5 6 7], [8 9 10 11].

N073 — S-এর শেষে নতুন student row যোগ

SOLUTION CODE

```
new_scores = np.vstack((S, [80, 85, 90]))
print(new_scores)
print("Original shape:", S.shape)
```

EXPLANATION (BANGLA)

vstack নতুন row যুক্ত করে নতুন array ফেরত দেয়; original S বদলায় না.

EXPECTED RESULT

নতুন shape=(5,3); S-এর shape=(4,3) থাকে.

N074 — S-এর শেষে fourth exam column যোগ

SOLUTION CODE

```
fourth_exam = np.array([88, 72, 94, 65])
updated = np.column_stack((S, fourth_exam))
print(updated.shape)
print(updated)
```

EXPLANATION (BANGLA)

column_stack 1D vector-কে column হিসেবে যুক্ত করে. এখানে 4টি student-এর জন্য 4টি score আছে.

EXPECTED RESULT

shape=(4,4).

N075 — stack axis=0 ও axis=1 তুলনা

SOLUTION CODE

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
s0 = np.stack((a, b), axis=0)
s1 = np.stack((a, b), axis=1)
print(s0, s0.shape)
print(s1, s1.shape)
```

EXPLANATION (BANGLA)

stack নতুন axis তৈরি করে. axis=0-তে দুইটি vector row হয়; axis=1-এ পাশাপাশি column হিসেবে বসে.

EXPECTED RESULT

axis=0 shape=(2,3); axis=1 shape=(3,2).

N076 — প্রতিটি element তিনবার repeat

SOLUTION CODE

```
arr = np.repeat([1, 2, 3], 3)
print(arr)
```

EXPLANATION (BANGLA)

repeat প্রতিটি element-কে নির্দিষ্ট সংখ্যকবার পাশাপাশি পুনরাবৃত্তি করে.

EXPECTED RESULT

[1 1 1 2 2 2 3 3 3].

N077 — [1,0] row-কে 4 বার tile

SOLUTION CODE

```
arr = np.tile([1, 0], (4, 1))
print(arr, arr.shape)
```

EXPLANATION (BANGLA)

tile(array,(4,1)) row axis-এ 4 বার এবং column axis-এ 1 বার repeat করে.

EXPECTED RESULT

shape=(4,2), প্রতিটি row [1,0].

N078 — 6x4 matrix row ও column অনুযায়ী split

SOLUTION CODE

```
matrix = np.arange(24).reshape(6, 4)
row_groups = np.split(matrix, 2, axis=0)
col_groups = np.split(matrix, 4, axis=1)
print([part.shape for part in row_groups])
print([part.shape for part in col_groups])
```

EXPLANATION (BANGLA)

axis=0 মানে row direction-এ ভাগ; axis=1 মানে column direction-এ ভাগ.

EXPECTED RESULT

দুই row-group-এর shape (3,4); চার column-group-এর প্রতিটির shape (6,1).

MODULE 08 | Vectorized arithmetic and NumPy ufuncs

12টি problem | N079 থেকে N090

N079 — A-এর প্রতিটি value-তে 5 যোগ

SOLUTION CODE

```
print(A + 5)
```

EXPLANATION (BANGLA)

Array + scalar broadcasting করে প্রতিটি element-এ 5 যোগ করে; loop লাগে না.

EXPECTED RESULT

[9 13 17 21 25].

N080 — B-এর প্রতিটি value 3 দিয়ে গুণ

SOLUTION CODE

```
print(B * 3)
```

EXPLANATION (BANGLA)

Array multiplication element-wise হয়; B-এর প্রতিটি value 3 দিয়ে গুণ হয়.

EXPECTED RESULT

[[6,12,18],[24,30,36],[42,48,54]].

N081 — দুই vector-এর add/subtract/multiply

SOLUTION CODE

```
a = np.array([2, 4, 6])
b = np.array([1, 3, 5])
print(a + b)
print(a - b)
print(a * b)
```

EXPLANATION (BANGLA)

সমান shape-এর vector-এ arithmetic element-wise হয়. প্রতিটি index-এ সংশ্লিষ্ট দুই value নিয়ে কাজ করে.

EXPECTED RESULT

যোগ [3,7,11]; বিয়োগ [1,1,1]; গুণ [2,12,30].

N082 — A-এর square ও cube

SOLUTION CODE

```
print(A ** 2)
print(A ** 3)
```

EXPLANATION (BANGLA)

** exponentiation operator. প্রতিটি element-এর power 2 বা 3 হিসাব হয়.

EXPECTED RESULT

Square [16,64,144,256,400]; cube [64,512,1728,4096,8000].

N083 — Square root

SOLUTION CODE

```
values = np.array([0, 1, 4, 9, 16])
print(np.sqrt(values))
```

EXPLANATION (BANGLA)

np.sqrt() প্রতিটি element-এর square root বের করে.

EXPECTED RESULT

[0. 1. 2. 3. 4.].

N084 — C-এর absolute value

SOLUTION CODE

```
print(np.abs(C))
```

EXPLANATION (BANGLA)

np.abs() negative value-এর sign সরিয়ে magnitude দেয়; zero ও positive value অপরিবর্তিত.

EXPECTED RESULT

[[4,0,3],[7,2,5]].

N085 — S থেকে 70 বিয়োগ

SOLUTION CODE

```
print(S - 70)
```

EXPLANATION (BANGLA)

Scalar broadcasting করে S-এর প্রতিটি score থেকে 70 বিয়োগ হয়.

EXPECTED RESULT

প্রতিটি score-এর 70 থেকে deviation পাওয়া যায়.

N086 — Base-10 logarithm

SOLUTION CODE

```
values = np.array([1, 10, 100, 1000])
logs = np.log10(values)
print(logs)
print(10 ** logs)
```

EXPLANATION (BANGLA)

log10 হলো 10 ভিত্তিক logarithm. 10-এর power 0,1,2,3 হওয়ায় ফল সেই exponent-গুলো.

EXPECTED RESULT

[0. 1. 2. 3.]; inverse করলে original values প্রায় ফিরে আসে.

N087 — exp ও log-এর সম্পর্ক

SOLUTION CODE

```
v = np.array([0, 1, 2])
e = np.exp(v)
back = np.log(e)
print(e)
print(back)
print(np.allclose(back, v))
```

EXPLANATION (BANGLA)

exp natural exponential e^x; log natural logarithm. তাত্ত্বিকভাবে log(exp(x))=x; floating precision-এর জন্য allclose ব্যবহার করা হয়.

EXPECTED RESULT

back প্রায় [0,1,2]; allclose=True.

N088 — Zero দিয়ে division ও finite check

SOLUTION CODE

```
a = np.array([10, 20, 30])
b = np.array([2, 0, 5])
with np.errstate(divide="ignore", invalid="ignore"):
    result = a / b
print(result)
print(np.isfinite(result))
```

EXPLANATION (BANGLA)

20/0 positive infinity হয়. np.errstate warning নিয়ন্ত্রণ করে, কিন্তু সমস্যাটি লুকায় না; isfinite() finite value চিহ্নিত করে.

EXPECTED RESULT

[5., inf, 6.]; finite mask [True, False, True].

N089 — Vectorized sigmoid function

SOLUTION CODE

```
def sigmoid(z):
    z = np.asarray(z, dtype=float)
    return 1 / (1 + np.exp(-z))

print(sigmoid(np.array([-4, 0, 4])))
```

EXPLANATION (BANGLA)

Sigmoid z-কে 0 থেকে 1-এর মধ্যে probability-like value-এ map করে. Array input-এ vectorizedভাবে কাজ করে.

EXPECTED RESULT

প্রায় [0.0180, 0.5, 0.9820].

N090 — Origin থেকে Euclidean distance

SOLUTION CODE

```
points = np.array([[0, 0], [3, 4], [6, 8]])
distances = np.linalg.norm(points, axis=1)
print(distances)
```

EXPLANATION (BANGLA)

প্রতিটি row একটি point. norm(axis=1) প্রতিটি row-এর sqrt(x^2+y^2) হিসাব করে.

EXPECTED RESULT

[0., 5., 10.].

MODULE 09 | Aggregation, axis and keepdims

14টি problem | N091 থেকে N104

N091 — B-এর সব element-এর sum

SOLUTION CODE

```
print(B.sum())
```

EXPLANATION (BANGLA)

axis না দিলে sum পুরো array-এর সব element যোগ করে.

EXPECTED RESULT

90.

N092 — A-এর mean ও maximum

SOLUTION CODE

```
print(A.mean())
print(A.max())
```

EXPLANATION (BANGLA)

mean average; max সর্বোচ্চ element দেয়.

EXPECTED RESULT

mean=12.0; max=20.

N093 — B-এর column-wise sum

SOLUTION CODE

```
print(B.sum(axis=0))
```

EXPLANATION (BANGLA)

axis=0 বরাবর row direction collapse হয়; প্রতিটি column-এর values যোগ হয়.

EXPECTED RESULT

[24 30 36].

N094 — B-এর row-wise sum

SOLUTION CODE

```
print(B.sum(axis=1))
```

EXPLANATION (BANGLA)

axis=1 বরাবর column direction collapse হয়; প্রতিটি row-এর values যোগ হয়.

EXPECTED RESULT

[12 30 48].

N095 — প্রতিটি student-এর average

SOLUTION CODE

```
student_avg = S.mean(axis=1)
print(student_avg)
```

EXPLANATION (BANGLA)

S-এর প্রতিটি row একজন student. axis=1 দিলে প্রতিটি row-এর তিন exam-এর average পাওয়া যায়.

EXPECTED RESULT

প্রায় [80.67, 70.00, 91.67, 59.00].

N096 — প্রতিটি exam-এর class average

SOLUTION CODE

```
exam_avg = S.mean(axis=0)
print(exam_avg)
```

EXPLANATION (BANGLA)

প্রতিটি column একটি exam. axis=0 দিলে সব student-এর ওই exam-এর average পাওয়া যায়.

EXPECTED RESULT

[73.00, 73.25, 79.75].

N097 — সেরা student-এর row index

SOLUTION CODE

```
averages = S.mean(axis=1)
best_index = np.argmax(averages)
print(averages, best_index)
```

EXPLANATION (BANGLA)

argmax() সর্বোচ্চ average-এর position দেয়; এটি 0-based row index.

EXPECTED RESULT

সেরা student row index 2; average প্রায় 91.67.

N098 — keepdims=True সহ row sum

SOLUTION CODE

```
row_sums = B.sum(axis=1, keepdims=True)
print(row_sums)
print(row_sums.shape)
```

EXPLANATION (BANGLA)

keepdims=True reduced axis-কে length 1 হিসেবে রেখে দেয়. তাই 3টি row sum-এর shape (3,1).

EXPECTED RESULT

[[12],[30],[48]], shape=(3,1).

N099 — X-এর feature-wise mean

SOLUTION CODE

```
print(X.mean(axis=0))
```

EXPLANATION (BANGLA)

X-এর প্রতিটি column একটি feature. axis=0 দিলে message length, URL count ও suspicious word count-এর আলাদা mean পাওয়া যায়.

EXPECTED RESULT

প্রায় [107.5, 1.0, 3.1667].

N100 — T-এর cumulative sum

SOLUTION CODE

```
cs = np.cumsum(T)
print(cs)
print("Last cumulative sum:", cs[-1])
```

EXPLANATION (BANGLA)

cumsum() চলমান যোগফল দেয়. শেষ cumulative sum সব value-এর মোট sum-এর সমান.

EXPECTED RESULT

[12,27,41,61,79,101,118]; শেষ value 118.

N101 — B-এর প্রতিটি row-এর product

SOLUTION CODE

```
print(np.prod(B, axis=1))
```

EXPLANATION (BANGLA)

prod(axis=1) প্রতিটি row-এর সব element গুণ করে.

EXPECTED RESULT

[48, 960, 4032].

N102 — S-এ 80 বা তার বেশি score count

SOLUTION CODE

```
mask = S >= 80
print(np.count_nonzero(mask))
print(np.count_nonzero(mask, axis=1))
```

EXPLANATION (BANGLA)

Comparison একটি Boolean matrix তৈরি করে. count_nonzero True value গোনে; axis=1 দিলে per-student count হয়.

EXPECTED RESULT

Overall 6; per student [3,0,3,0].

N103 — B-এর প্রতিটি column mean বাদ

SOLUTION CODE

```
column_means = B.mean(axis=0, keepdims=True)
centered = B - column_means
print(centered)
print(centered.mean(axis=0))
```

EXPLANATION (BANGLA)

keepdims=True column mean-কে shape (1,3) রাখে. Broadcasting করে প্রতিটি row থেকে সংশ্লিষ্ট column mean বাদ হয়.

EXPECTED RESULT

Centered matrix-এর প্রতিটি column mean প্রায় zero.

N104 — S-এর প্রতিটি row mean বাদ

SOLUTION CODE

```
row_means = S.mean(axis=1, keepdims=True)
centered = S - row_means
print(centered)
print(centered.mean(axis=1))
```

EXPLANATION (BANGLA)

প্রতিটি student-এর row mean (n,1) shape-এ রাখা হয়. Broadcasting করে ওই student-এর সব exam score থেকে তার average বাদ হয়.

EXPECTED RESULT

প্রতিটি row-এর mean প্রায় zero.

MODULE 10 | Broadcasting and shape compatibility

12টি problem | N105 থেকে N116

N105 — A-তে scalar 10 যোগ

SOLUTION CODE

```
result = A + 10
print(result)
```

EXPLANATION (BANGLA)

Scalar 10 broadcast হয়ে A-এর প্রতিটি element-এর সঙ্গে যোগ হয়. Original A পরিবর্তিত হয় না; নতুন result তৈরি হয়.

EXPECTED RESULT

[14 18 22 26 30].

N106 — B-এর প্রতিটি row-তে row vector যোগ

SOLUTION CODE

```
row = np.array([1, 2, 3])
result = B + row
print(result)
```

EXPLANATION (BANGLA)

B-এর shape (3,3), row vector-এর shape (3,). শেষ dimension মিলে যাওয়ায় row vector প্রতিটি row-তে broadcast হয়.

EXPECTED RESULT

[[3,6,9],[9,12,15],[15,18,21]].

N107 — প্রতিটি row থেকে column vector বিয়োগ

SOLUTION CODE

```
column = np.array([[1], [2], [3]])
result = B - column
print(result)
```

EXPLANATION (BANGLA)

Column vector-এর shape (3,1). প্রতিটি row-এর জন্য একটি করে value broadcast হয়ে ওই row-এর সব column থেকে বাদ যায়.

EXPECTED RESULT

[[1,3,5],[6,8,10],[11,13,15]].

N108 — 3x4 matrix scalar দিয়ে multiply

SOLUTION CODE

```
matrix = np.arange(12).reshape(3, 4)
result = matrix * 2
print(result.shape)
print(result)
```

EXPLANATION (BANGLA)

Scalar multiplication shape বদলায় না; প্রতিটি element 2 দিয়ে গুণ হয়.

EXPECTED RESULT

Output shape=(3,4).

N109 — (1,3) ও (4,3) shape যোগ

SOLUTION CODE

```
a = np.array([[1, 2, 3]])
b = np.ones((4, 3), dtype=int)
result = a + b
print(a.shape, b.shape, result.shape)
print(result)
```

EXPLANATION (BANGLA)

প্রথম array-এর row axis length 1, তাই চার row-এ broadcast হয়. শেষ dimension 3 দুই দিকেই সমান.

EXPECTED RESULT

Input shapes (1,3), (4,3); output shape (4,3).

N110 — (4,1) ও (4,3) যোগ

SOLUTION CODE

```
a = np.arange(1, 5).reshape(4, 1)
b = np.ones((4, 3), dtype=int)
result = a + b
print(result.shape)
print(result)
```

EXPLANATION (BANGLA)

প্রতিটি row-এর একটি value column direction-এ তিনবার broadcast হয়.

EXPECTED RESULT

shape=(4,3); row values যথাক্রমে 2,3,4,5 দিয়ে পূর্ণ

N111 — দুই tax rate দিয়ে তিন price-এর taxed matrix

SOLUTION CODE

```
prices = np.array([100, 200, 300])
tax_rates = np.array([0.05, 0.10])
taxed = prices * (1 + tax_rates[:, np.newaxis])
print(taxed, taxed.shape)
```

EXPLANATION (BANGLA)

tax_rates[:,None] shape (2,1) হয়; prices shape (3,). Broadcasting-এ 2টি tax rate ও 3টি price মিলিয়ে (2,3) matrix তৈরি হয়.

EXPECTED RESULT

5% row [105,210,315]; 10% row [110,220,330]; shape=(2,3).

N112 — B-এর প্রতিটি column নিজস্ব maximum দিয়ে normalize

SOLUTION CODE

```
column_max = B.max(axis=0, keepdims=True)
normalized = B / column_max
print(normalized)
print(normalized.max(axis=0))
```

EXPLANATION (BANGLA)

axis=0-তে column maximum বের হয়. keepdims shape (1,3) রাখে; broadcasting করে প্রতিটি column তার নিজের maximum দিয়ে ভাগ হয়.

EXPECTED RESULT

প্রতিটি column-এর maximum 1.0 হয়.

N113 — (5,1,3) ও (1,4,3) broadcast

SOLUTION CODE

```
a = np.zeros((5, 1, 3))
b = np.zeros((1, 4, 3))
print((a + b).shape)
```

EXPLANATION (BANGLA)

ডান দিক থেকে dimension মিলিয়ে দেখি: 3=3, 1 বনাম 4 broadcast হয়, 5 বনাম 1 broadcast হয়.

EXPECTED RESULT

Result shape=(5,4,3).

N114 — (3,2) ও (3,) broadcast error

SOLUTION CODE

```
a = np.zeros((3, 2))
b = np.zeros(3)
try:
    print(a + b)
except ValueError as error:
    print("Shape error:", error)
```

EXPLANATION (BANGLA)

ডান দিক থেকে compare করলে (3,2) ও (1,3): শেষ dimension 2 ও 3; কোনোটিই 1 নয়' তাই incompatible.

EXPECTED RESULT

ValueError; shapes broadcast করা যায় না.

N115 — X-এর feature-wise standardization

SOLUTION CODE

```
mean = X.mean(axis=0)
std = X.std(axis=0)
safe_std = np.where(std == 0, 1, std)
standardized = (X - mean) / safe_std
print(standardized)
print(standardized.mean(axis=0))
print(standardized.std(axis=0))
```

EXPLANATION (BANGLA)

প্রতিটি feature-এর mean ও standard deviation আলাদা হিসাব হয়. std শূন্য হলে 1 দিয়ে divide করা হয়’ যাতে zero-division না হয়·

EXPECTED RESULT

প্রতিটি column-এর mean প্রায় 0 এবং population std প্রায় 1.

N116 — (2,3) ও (3,2) যোগের error

SOLUTION CODE

```
a = np.zeros((2, 3))
b = np.zeros((3, 2))
try:
    result = a + b
except ValueError as error:
    print("Cannot add:", error)
```

EXPLANATION (BANGLA)

Broadcasting-এ aligned dimension equal হতে হবে অথবা কোনো একটি 1 হতে হবে· এখানে 3 বনাম 2 mismatch, তাই যোগ সম্ভব নয়·

EXPECTED RESULT

ValueError ধরা পড়ে·

MODULE 11 | Comparisons and Boolean masks

12টি problem | N117 থেকে N128

N117 — A-এর 10-এর বেশি value mask

SOLUTION CODE

```
mask = A > 10
print(mask)
```

EXPLANATION (BANGLA)

Comparison প্রতিটি element-এ condition পরীক্ষা করে Boolean array দেয়.

EXPECTED RESULT

[False False True True True].

N118 — A থেকে even value select

SOLUTION CODE

```
even_mask = A % 2 == 0
print(A[even_mask])
```

EXPLANATION (BANGLA)

Modulo 2-এর remainder zero হলে value even. Boolean mask দিয়ে শুধু True position-এর value নেওয়া হয়.

EXPECTED RESULT

[4 8 12 16 20].

N119 — S-এ 60-এর নিচে score count

SOLUTION CODE

```
below_pass = S < 60
print(np.count_nonzero(below_pass))
```

EXPLANATION (BANGLA)

S<60 Boolean matrix তৈরি করে; count_nonzero() True value গোনে.

EXPECTED RESULT

1টি score (55).

N120 — C থেকে negative value select

SOLUTION CODE

```
print(C[C < 0])
```

EXPLANATION (BANGLA)

Boolean indexing matrix flatten করে condition পূরণ করা value-গুলো ফেরত দেয়.

EXPECTED RESULT

[-4 -2].

N121 — URL count positive এমন X row

SOLUTION CODE

```
url_rows = X[X[:, 1] > 0]
print(url_rows)
print(url_rows.shape)
```

EXPLANATION (BANGLA)

X[:,1] সব row-এর URL count column. >0 mask তৈরি করে সংশ্লিষ্ট সম্পূর্ণ row select করা হয়.

EXPECTED RESULT

Row index 0,2,3; shape=(3,3).

N122 — Length>100 AND URL>=1 row

SOLUTION CODE

```
mask = (X[:, 0] > 100) & (X[:, 1] >= 1)
print(X[mask])
```

EXPLANATION (BANGLA)

দুইটি condition-এ element-wise AND করতে & ব্যবহার হয়; প্রতিটি comparison bracket-এ রাখতে হয়.

EXPECTED RESULT

প্রথম ও তৃতীয় sample select হয়.

N123 — Length<50 OR suspicious count>=5

SOLUTION CODE

```
mask = (X[:, 0] < 50) | (X[:, 2] >= 5)
print(X[mask])
```

EXPLANATION (BANGLA)

OR condition-এর জন্য | ব্যবহার হয়· যেকোনো একটি condition True হলেই row select হয়·

EXPECTED RESULT

Row index 1,2,4,5; shape=(4,3).

N124 — logical_not দিয়ে mask invert

SOLUTION CODE

```
above_10 = A > 10
not_above_10 = np.logical_not(above_10)
print(not_above_10)
```

EXPLANATION (BANGLA)

logical_not প্রতিটি Boolean value উল্টে দেয়: True→False এবং False→True.

EXPECTED RESULT

[True True False False False].

N125 — y-তে scam ও non-scam count

SOLUTION CODE

```
print("scam:", np.count_nonzero(y == 1))
print("non-scam:", np.count_nonzero(y == 0))
```

EXPLANATION (BANGLA)

Label 1 ও label 0-এর জন্য আলাদা equality mask তৈরি করে True count করা হয়·

EXPECTED RESULT

scam=3, non-scam=3.

N126 — y==1 হলে X row select

SOLUTION CODE

```
scam_X = X[y == 1]
print(scam_X)
print(scam_X.shape)
```

EXPLANATION (BANGLA)

y==1 mask sample axis-এ X-এর row select করে. X ও y-এর sample order aligned থাকতে হবে·

EXPECTED RESULT

Row 0,2,4; shape=(3,3).

N127 — প্রতিটি student সব exam-এ 60 বা বেশি পেয়েছে কি না

SOLUTION CODE

```
passed_all = np.all(S >= 60, axis=1)
print(passed_all)
```

EXPLANATION (BANGLA)

S>=60 element-wise Boolean matrix. np.all(axis=1) প্রতিটি row-এর সব exam True কি না পরীক্ষা করে·

EXPECTED RESULT

[True True True False].

N128 — URL নেই কিন্তু suspicious word আছে এমন row index

SOLUTION CODE

```
mask = (X[:, 1] == 0) & (X[:, 2] > 0)
print(np.where(mask)[0])
```

EXPLANATION (BANGLA)

URL count 0 এবং suspicious word count 0-এর বেশি — দুই শর্ত একসঙ্গে প্রয়োগ হয়. np.where()[0] matching row index দেয়·

EXPECTED RESULT

Index [4 5].

MODULE 12 | Conditional transforms and clipping

10টি problem | N129 থেকে N138

N129 — C-এর negative value 0 দিয়ে replace

SOLUTION CODE

```
clean = np.maximum(C, 0)
print(clean)
```

EXPLANATION (BANGLA)

np.maximum দুই input-এর element-wise maximum নেয়. Scalar 0-এর সঙ্গে তুলনায় negative value 0 হয়; positive value থাকে.

EXPECTED RESULT

[[0,0,3],[7,0,5]].

N130 — Value-কে 0-100 range-এ clamp

SOLUTION CODE

```
values = np.array([-10, 2, 20, 110])
print(np.clip(values, 0, 100))
```

EXPLANATION (BANGLA)

clip lower ও upper boundary নির্ধারণ করে. 0-এর নিচে 0, 100-এর ওপরে 100; মাঝের value অপরিবর্তিত.

EXPECTED RESULT

[0 2 20 100].

N131 — Student average থেকে Pass/Fail label

SOLUTION CODE

```
avg = S.mean(axis=1)
labels = np.where(avg >= 60, "Pass", "Fail")
print(avg)
print(labels)
```

EXPLANATION (BANGLA)

প্রথমে row-wise average; তারপর np.where condition অনুযায়ী text label দেয়. Threshold 60 inclusive.

EXPECTED RESULT

প্রথম তিনজন Pass, চতুর্থজন Fail.

N132 — প্রতিটি sample-এ URL আছে কি না 0/1 feature

SOLUTION CODE

```
has_url = (X[:, 1] > 0).astype(int)
print(has_url)
```

EXPLANATION (BANGLA)

URL count positive হলে Boolean True; astype(int) True-কে 1 এবং False-কে 0 করে.

EXPECTED RESULT

[1 0 1 1 0 0].

N133 — S-এর 60-এর নিচের value 60 করা; original অক্ষত

SOLUTION CODE

```
adjusted = np.maximum(S, 60)
print(adjusted)
print("Original minimum:", S.min())
```

EXPLANATION (BANGLA)

np.maximum নতুন result তৈরি করে. S-এ সরাসরি assignment হয়নি' তাই original S অক্ষত থাকে.

EXPECTED RESULT

55-এর জায়গায় 60; অন্য score একই.

N134 — A-কে low/medium/high bucket

SOLUTION CODE

```
labels = np.select(
    [A < 10, (A >= 10) & (A <= 15), A > 15],
    ["low", "medium", "high"],
    default="unknown"
)
print(labels)
```

EXPLANATION (BANGLA)

np.select একাধিক condition ও corresponding choice নেয়. Condition-এর order অনুযায়ী label assign হয়.

EXPECTED RESULT

['low','low','medium','high','high'].

N135 — Score থেকে A/B/C/F grade

SOLUTION CODE

```
scores = np.array([55, 60, 72, 85, 90])
grades = np.select(
    [scores >= 80, scores >= 70, scores >= 60],
    ["A", "B", "C"],
    default="F"
)
print(grades)
```

EXPLANATION (BANGLA)

Condition-গুলো বড় threshold থেকে ছোট threshold-এ রাখা হয়েছে' কোনো condition True না হলে default F.

EXPECTED RESULT

['F','C','B','A','A'].

N136 — শুধু condition পূরণ হলে square

SOLUTION CODE

```
matrix = np.array([[-2, 1], [3, 4]])
result = np.where(matrix > 0, matrix ** 2, matrix)
print(result)
```

EXPLANATION (BANGLA)

np.where(condition, x, y) condition True হলে square value, False হলে original value রাখে'

EXPECTED RESULT

[[-2,1],[9,16]].

N137 — Zero থাকলে safe reciprocal NaN

SOLUTION CODE

```
values = np.array([2, 0, 4, 0, 8], dtype=float)
reciprocal = np.divide(
    1.0, values,
    out=np.full(values.shape, np.nan),
    where=values != 0
)
print(reciprocal)
```

EXPLANATION (BANGLA)

divide-এর where mask শুধু nonzero position-এ ভাগ করে' আগে output-এ NaN রাখা হয়' তাই zero position NaN-ই থাকে'

EXPECTED RESULT

[0.5, nan, 0.25, nan, 0.125].

N138 — 5th ও 95th percentile winsorization

SOLUTION CODE

```
values = np.array([1, 2, 3, 4, 5, 100], dtype=float)
low, high = np.percentile(values, [5, 95])
winsorized = np.clip(values, low, high)
print(low, high)
print(winsorized)
```

EXPLANATION (BANGLA)

প্রথমে lower ও upper percentile boundary বের হয়. clip boundary-এর বাইরে value-গুলোকে boundary-তে cap করে; row delete করে না'

EXPECTED RESULT

1 ও 100 percentile boundary-তে cap হয়; মাঝের value অপরিবর্তিত'

MODULE 13 | Sort, find, rank and count unique values

12টি problem | N139 থেকে N150

N139 — Ascending sort

SOLUTION CODE

```
arr = np.array([9, 1, 5, 1, 7])
print(np.sort(arr))
```

EXPLANATION (BANGLA)

np.sort() ছোট থেকে বড় order-এ নতুন sorted array দেয়; original array বদলায় না।

EXPECTED RESULT

[1 1 5 7 9].

N140 — Descending sort

SOLUTION CODE

```
arr = np.array([9, 1, 5, 1, 7])
print(np.sort(arr)[::-1])
```

EXPLANATION (BANGLA)

প্রথমে ascending sort; তারপর [::-1] দিয়ে order reverse করা হয়।

EXPECTED RESULT

[9 7 5 1 1].

N141 — A-এর largest ও smallest value-এর index

SOLUTION CODE

```
print("largest index:", np.argmax(A))
print("smallest index:", np.argmin(A))
```

EXPLANATION (BANGLA)

argmax/argmin value নয়, value-এর position ফেরত দেয়।

EXPECTED RESULT

largest index=4; smallest index=0.

N142 — Z-এর unique value ও count

SOLUTION CODE

```
unique_values = np.unique(Z)
print(unique_values)
print(unique_values.size)
```

EXPLANATION (BANGLA)

np.unique duplicate সরিয়ে sorted distinct value ফেরত দেয়. size দিয়ে distinct count পাওয়া যায়।

EXPECTED RESULT

[1 2 3 4 5], distinct count=5.

N143 — Z-এর প্রতিটি value-এর frequency

SOLUTION CODE

```
values, counts = np.unique(Z, return_counts=True)
print(values)
print(counts)
```

EXPLANATION (BANGLA)

return_counts=True প্রতিটি unique value কতবার এসেছে তা একই order-এ ফেরত দেয়।

EXPECTED RESULT

values=[1,2,3,4,5]; counts=[1,2,3,1,2].

N144 — B-এর row-wise ও column-wise sort

SOLUTION CODE

```
print("Row sort:\n", np.sort(B, axis=1))
print("Column sort:\n", np.sort(B, axis=0))
```

EXPLANATION (BANGLA)

axis=1 প্রতিটি row আলাদা sort করে. axis=0 প্রতিটি column আলাদা sort করে।

EXPECTED RESULT

B বর্তমানে row ও column উভয় দিকেই ascending; তাই output-এ B-এর মতোই দেখাবে।

N145 — [40,10,30,20] sort করতে প্রয়োজনীয় index

SOLUTION CODE

```
arr = np.array([40, 10, 30, 20])
indices = np.argsort(arr)
print(indices)
print(arr[indices])
```

EXPLANATION (BANGLA)

argsort sorted value নয়, sorted order পেতে original array-এর index ফেরত দেয়.

EXPECTED RESULT

Indices [1,3,2,0]; sorted values [10,20,30,40].

N146 — X-এর top 3 message length-এর index

SOLUTION CODE

```
lengths = X[:, 0]
top_indices = np.argsort(lengths)[-3:][::-1]
print(top_indices)
print(lengths[top_indices])
```

EXPLANATION (BANGLA)

argsort ascending order-এর index দেয়. শেষ তিনটি নিয়ে reverse করলে largest থেকে smallest পাওয়া যায়.

EXPECTED RESULT

Indices [2,4,0]; lengths [200,160,120].

N147 — S-এর top 3 exam average ও student index

SOLUTION CODE

```
averages = S.mean(axis=1)
top_indices = np.argsort(averages)[-3:][::-1]
print(averages[top_indices])
print(top_indices)
```

EXPLANATION (BANGLA)

প্রথমে row-wise average; এরপর descending rank-এর index নেওয়া হয়.

EXPECTED RESULT

Top averages প্রায় [91.67,80.67,70]; indices [2,0,1].

N148 — bincount ও unique count তুলনা

SOLUTION CODE

```
print(np.bincount(y))
print(np.unique(y, return_counts=True))
```

EXPLANATION (BANGLA)

bincount non-negative integer label-এর frequency সরাসরি index position-এ রাখে. unique return_counts general unique values ও count দেয়.

EXPECTED RESULT

দুই পদ্ধতিতেই label 0 ও 1-এর count [3,3].

N149 — T-তে প্রথম value >=20-এর index

SOLUTION CODE

```
matches = np.flatnonzero(T >= 20)
first_index = matches[0] if matches.size else None
print(first_index)
```

EXPLANATION (BANGLA)

flatnonzero condition পূরণ করা index দেয়. Empty হলে প্রথম index access না করে None রাখা হয়েছে.

EXPECTED RESULT

প্রথম matching index=3.

N150 — Duplicate row বাদ দিয়ে unique rows

SOLUTION CODE

```
rows = np.array([[1, 2], [1, 2], [2, 3], [3, 4], [2, 3]])
unique_rows = np.unique(rows, axis=0)
print(unique_rows)
```

EXPLANATION (BANGLA)

axis=0 দিয়ে row-wise uniqueness পরীক্ষা হয়. Duplicate row একবার রাখা হয়; output lexicographically sorted হতে পারে.

EXPECTED RESULT

[[1,2],[2,3],[3,4]].

MODULE 14 | Descriptive statistics for research

12টি problem | N151 থেকে N162

N151 — T-এর mean, median, min, max

SOLUTION CODE

```
print(np.mean(T), np.median(T), np.min(T), np.max(T))
```

EXPLANATION (BANGLA)

Mean average; median sorted data-এর middle; min ও max range-এর দুই প্রান্ত.

EXPECTED RESULT

Mean≈16.8571, median=17, min=12, max=22.

N152 — S-এর প্রতিটি column-এর range

SOLUTION CODE

```
column_ranges = S.max(axis=0) - S.min(axis=0)
print(column_ranges)
```

EXPLANATION (BANGLA)

প্রতিটি exam column-এর maximum থেকে minimum বাদ দিলে range পাওয়া যায়.

EXPECTED RESULT

[35 33 30].

N153 — T-এর population variance ও standard deviation

SOLUTION CODE

```
print(np.var(T, ddof=0))
print(np.std(T, ddof=0))
```

EXPLANATION (BANGLA)

ddof=0 population formula ব্যবহার করে. Variance হলো squared deviation-এর mean; standard deviation তার square root.

EXPECTED RESULT

Population variance=10.4082; population standard deviation=3.2262.

N154 — T-এর quartile

SOLUTION CODE

```
q1, q2, q3 = np.percentile(T, [25, 50, 75])
print(q1, q2, q3)
```

EXPLANATION (BANGLA)

Percentile data distribution-এর অবস্থান বোঝায়. 25th=Q1, 50th=median, 75th=Q3.

EXPECTED RESULT

Q1=14.5, median=17, Q3=19.

N155 — Student average ও overall class average

SOLUTION CODE

```
student_avg = S.mean(axis=1)
class_avg = S.mean()
print(student_avg)
print(class_avg)
```

EXPLANATION (BANGLA)

axis=1 student-wise average; axis না দিলে পুরো score matrix-এর overall mean.

EXPECTED RESULT

Student average প্রায় [80.67,70,91.67,59]; overall mean=75.25.

N156 — Sample standard deviation ও ddof

SOLUTION CODE

```
print("population:", T.std(ddof=0))
print("sample:", T.std(ddof=1))
```

EXPLANATION (BANGLA)

ddof=1 sample standard deviation-এ denominator n-1 হয়; sample থেকে population variability estimate করার সময় এটি প্রচলিত.

EXPECTED RESULT

Sample std≈3.4847; population std≈3.2262.

N157 — Message length-এর 25th ও 75th percentile

SOLUTION CODE

```
lengths = X[:, 0]
print(np.percentile(lengths, [25, 75]))
```

EXPLANATION (BANGLA)

X[:,0] message length feature. Percentile দিয়ে lower ও upper quartile position পাওয়া যায়.

EXPECTED RESULT

25th percentile=56.25; 75th percentile=150.

N158 — Weighted average

SOLUTION CODE

```
scores = np.array([80, 70, 90])
weights = np.array([0.2, 0.3, 0.5])
print(np.average(scores, weights=weights))
```

EXPLANATION (BANGLA)

Weighted average-এ প্রতিটি score তার weight দিয়ে গুণ হয়; weighted sum-কে total weight দিয়ে ভাগ করা হয়.

EXPECTED RESULT

81.0.

N159 — B-এর column median ও overall median

SOLUTION CODE

```
print(np.median(B, axis=0))
print(np.median(B))
```

EXPLANATION (BANGLA)

axis=0 column-wise median দেয়; axis বাদ দিলে সব element একসঙ্গে নিয়ে median দেয়.

EXPECTED RESULT

Column median [8,10,12]; overall median=10.

N160 — NumPy দিয়ে 3-bin histogram

SOLUTION CODE

```
counts, edges = np.histogram(T, bins=3)
print("counts:", counts)
print("edges:", edges)
```

EXPLANATION (BANGLA)

histogram data-কে তিনটি interval/bin-এ ভাগ করে প্রতিটি bin-এ কয়টি value আছে তা গোনে. Matplotlib প্রয়োজন নেই.

EXPECTED RESULT

Counts সাধারণত [3,2,2]; edges 12 থেকে 22 পর্যন্ত তিনটি bin boundary.

N161 — X-এর covariance matrix

SOLUTION CODE

```
cov = np.cov(X, rowvar=False)
print(cov)
print(cov.shape)
```

EXPLANATION (BANGLA)

rowvar=False মানে column-গুলো variable/feature. Covariance matrix প্রতিটি feature pair-এর covariance রাখে.

EXPECTED RESULT

Shape=(3,3); covariance matrix প্রায় [[4337.5,57,169.5],[57,1.6,2.4],[169.5,2.4,6.9667]].

N162 — Message length ও suspicious word count correlation

SOLUTION CODE

```
corr = np.corrcoef(X[:, 0], X[:, 2])[0, 1]
print(corr)
# Correlation does not prove causation.
```

EXPLANATION (BANGLA)

corrcoef দুই feature-এর Pearson correlation matrix দেয়. [0,1] position-এ দুই variable-এর correlation coefficient.

EXPECTED RESULT

Pearson correlation≈0.9751 (এই ছোট fictional data-তে strong positive association); correlation causation প্রমাণ করে না.

MODULE 15 | Random numbers and reproducible experiments

12টি problem | N163 থেকে N174

N163 — Seed 42 দিয়ে Generator তৈরি

SOLUTION CODE

```
rng = np.random.default_rng(42)
print(type(rng))
```

EXPLANATION (BANGLA)

default_rng() হলো NumPy-এর recommended random generator API. Seed 42 একই environment/API-তে reproducible random sequence দেয়.

EXPECTED RESULT

Generator object তৈরি হয়.

N164 — 0 থেকে 9 inclusive random integer

SOLUTION CODE

```
rng = np.random.default_rng(42)
values = rng.integers(0, 10, size=10)
print(values)
```

EXPLANATION (BANGLA)

integers-এর high bound exclusive; তাই 0-9 পেতে high=10. size=10 মানে 10টি integer.

EXPECTED RESULT

১০টি reproducible integer, প্রতিটি 0-9 range-এ.

N165 — 3x4 random float matrix

SOLUTION CODE

```
rng = np.random.default_rng(42)
values = rng.random((3, 4))
print(values)
print(values.shape)
```

EXPLANATION (BANGLA)

random() uniform distribution থেকে [0,1) range-এর float তৈরি করে.

EXPECTED RESULT

shape=(3,4); সব value 0 বা তার বেশি এবং 1-এর কম.

N166 — 500 standard normal value

SOLUTION CODE

```
rng = np.random.default_rng(42)
values = rng.standard_normal(500)
print(values.mean())
print(values.std(ddof=1))
```

EXPLANATION (BANGLA)

standard_normal mean 0 ও standard deviation 1-এর theoretical normal distribution থেকে sample নেয়. ddof=1 sample std দেয়.

EXPECTED RESULT

Sample mean প্রায় 0 এবং sample std প্রায় 1; exact result random seed-নির্ভর.

N167 — Mean 100 ও std 15 normal sample

SOLUTION CODE

```
rng = np.random.default_rng(42)
values = rng.normal(loc=100, scale=15, size=5)
print(values)
```

EXPLANATION (BANGLA)

loc distribution-এর mean; scale standard deviation; size কয়টি sample তৈরি হবে.

EXPECTED RESULT

পাঁচটি reproducible normal random value, সাধারণত 100-এর আশেপাশে.

N168 — Same seed বনাম different seed

SOLUTION CODE

```
a = np.random.default_rng(42).random(5)
b = np.random.default_rng(42).random(5)
c = np.random.default_rng(7).random(5)
print(np.array_equal(a, b))
print(np.array_equal(a, c))
```

EXPLANATION (BANGLA)

একই seed ও একই call sequence একই result দেয়. Seed বদলালে সাধারণত sequence বদলে যায়.

EXPECTED RESULT

প্রথম comparison True; দ্বিতীয়টি False.

N169 — Original না বদলে permutation

SOLUTION CODE

```
rng = np.random.default_rng(42)
original = np.arange(10)
shuffled = rng.permutation(original)
print(shuffled)
print(original)
```

EXPLANATION (BANGLA)

permutation shuffled copy/order দেয়; input original array অপরিবর্তিত থাকে.

EXPECTED RESULT

একটি shuffled 0-9 sequence; original [0,1,...,9] থাকে.

N170 — A থেকে replacement ছাড়া 4টি sample

SOLUTION CODE

```
rng = np.random.default_rng(42)
sample = rng.choice(A, size=4, replace=False)
print(sample)
```

EXPLANATION (BANGLA)

replace=False মানে একই element দ্বিতীয়বার sample হতে পারবে না. A-তে পাঁচটি value, তাই চারটি নেওয়া সম্ভব.

EXPECTED RESULT

A থেকে চারটি distinct value পাওয়া যাবে.

N171 — 1000 fair coin flip simulation

SOLUTION CODE

```
rng = np.random.default_rng(42)
flips = rng.integers(0, 2, size=1000)
head_fraction = np.mean(flips == 1)
print(head_fraction)
```

EXPLANATION (BANGLA)

0 ও 1 সমান probability-তে generate করা হয়েছে. 1-কে head ধরে mean of Boolean mask head fraction দেয়.

EXPECTED RESULT

Fraction 0.5-এর কাছাকাছি হবে; exact value random.

N172 — X row index-এর reproducible 80/20 split

SOLUTION CODE

```
rng = np.random.default_rng(42)
indices = rng.permutation(len(X))
cut = int(0.8 * len(X))
train_idx, test_idx = indices[:cut], indices[cut:]
print(train_idx, test_idx)
print(len(train_idx), len(test_idx))
```

EXPLANATION (BANGLA)

প্রথমে row index shuffle; তারপর 80% ও 20% ভাগ. X এবং y-কে একই index দিয়ে select করতে হবে.

EXPECTED RESULT

৬ sample-এর ক্ষেত্রে train=4, test=2.

N173 — Bootstrap mean ও percentile interval

SOLUTION CODE

```
def bootstrap_mean_ci(arr, rng, repeats=1000):
    arr = np.asarray(arr, dtype=float)
    means = np.empty(repeats)
    for i in range(repeats):
        sample = rng.choice(arr, size=arr.size, replace=True)
        means[i] = sample.mean()
    return means.mean(), np.percentile(means, [2.5, 97.5])

rng = np.random.default_rng(42)
print(bootstrap_mean_ci(T, rng))
```

EXPLANATION (BANGLA)

Bootstrap-এ replacement সহ original sample size-এর resample তৈরি হয়. প্রতিটি resample-এর mean জমা রেখে percentile interval হিসাব করা হয়.

EXPECTED RESULT

Estimated bootstrap mean এবং 95% percentile interval print হবে; seed দিলে reproducible.

N174 — একই permutation দিয়ে X ও y shuffle

SOLUTION CODE

```
rng = np.random.default_rng(42)
idx = rng.permutation(len(X))
X_shuffled = X[idx]
y_shuffled = y[idx]
assert np.array_equal(X_shuffled, X[idx])
assert np.array_equal(y_shuffled, y[idx])
print(X_shuffled.shape, y_shuffled.shape)
```

EXPLANATION (BANGLA)

Feature row ও label একই idx দিয়ে reorder করা জরুরি. আলাদা shuffle করলে sample-label alignment নষ্ট হবে.

EXPECTED RESULT

X shape=(6,3), y shape=(6,); alignment একই permutation-এ বজায় থাকে.

MODULE 16 | Views, copies and mutability

10টি problem | N175 থেকে N184

N175 — copy() modify করলে original অক্ষত

SOLUTION CODE

```
b = A.copy()
b[0] = 999
print("b:", b)
print("A:", A)
```

EXPLANATION (BANGLA)

copy() আলাদা memory তৈরি করে. b বদলালেও A-এর value বদলায় না.

EXPECTED RESULT

b-এর first value 999; A-এর first value 4 থাকে.

N176 — Copy ছাড়া alias তৈরি

SOLUTION CODE

```
b = A
b[0] = 999
print("b:", b)
print("A:", A)
```

EXPLANATION (BANGLA)

b=A করলে নতুন data copy হয় না; b ও A একই array object-কে refer করে.

EXPECTED RESULT

b[0] বদলালে A[0]-ও 999 হয়.

N177 — B slice view modify

SOLUTION CODE

```
sub = B[:, :2]
sub[0, 0] = -99
print(B)
```

EXPLANATION (BANGLA)

Basic slice সাধারণত view. sub-এর প্রথম value বদলালে B-এর একই memory location বদলায়.

EXPECTED RESULT

B[0,0] হয় -99.

N178 — B slice-এর independent copy

SOLUTION CODE

```
sub_copy = B[:, :2].copy()
sub_copy[0, 0] = -99
print("Copy:", sub_copy)
print("Original B:", B)
```

EXPLANATION (BANGLA)

copy() slice-এর data আলাদা memory-তে নেয়. তাই copy modify করলেও original B অপরিবর্তিত.

EXPECTED RESULT

sub_copy বদলায়; B বদলায় না.

N179 — B ও B.T memory share করে কি না

SOLUTION CODE

```
print(np.shares_memory(B, B.T))
```

EXPLANATION (BANGLA)

Transpose সাধারণত একই underlying data-কে অন্য stride/order দিয়ে view হিসেবে দেখায়.

EXPECTED RESULT

True.

N180 — flatten ও ravel-এর memory sharing পরীক্ষা

SOLUTION CODE

```
base = B.copy()
r = base.ravel()
f = base.flatten()
r[0] = -10
f[1] = -20
print(base)
print("shares ravel:", np.shares_memory(base, r))
print("shares flatten:", np.shares_memory(base, f))
```

EXPLANATION (BANGLA)

Contiguous B-এর ravel সাধারণত view, flatten copy. তাই ravel-এর change base-এ যায়; flatten-এর change যায় না।

EXPECTED RESULT

shares ravel=True; shares flatten=False.

N181 — Integer fancy indexing copy কি না

SOLUTION CODE

```
selected = A[[0, 2, 4]]
selected[0] = -100
print(selected)
print(A)
```

EXPLANATION (BANGLA)

Integer array/list indexing হলো fancy indexing; এটি copy ফেরত দেয়।

EXPECTED RESULT

selected বদলায়, original A অপরিবর্তিত।

N182 — Boolean mask selection copy কি না

SOLUTION CODE

```
selected = A[A > 10]
selected[0] = -100
print(selected)
print(A)
```

EXPLANATION (BANGLA)

Boolean indexing-ও নতুন array/copy ফেরত দেয়. Selected array বদলালে original বদলায় না।

EXPECTED RESULT

A অপরিবর্তিত থাকে।

N183 — Input mutate না করে safe normalization

SOLUTION CODE

```
def safe_normalize(arr):
    x = np.asarray(arr, dtype=float).copy()
    low, high = x.min(), x.max()
    if high == low:
        return np.zeros_like(x)
    return (x - low) / (high - low)

original = np.array([2., 4., 6.])
saved = original.copy()
normalized = safe_normalize(original)
print(normalized)
print(np.array_equal(original, saved))
```

EXPLANATION (BANGLA)

Function শুরুতেই copy নেয়. Min-max formula প্রয়োগ করে; constant array হলে zero division এড়াতে zeros ফেরত দেয়।

EXPECTED RESULT

Normalized [0,0.5,1]; original unchanged check True.

N184 — Successive slice ও copy-এর প্রয়োজন

SOLUTION CODE

```
arr = np.arange(10)
first = arr[1:]
second = first[::2]
print(np.shares_memory(arr, first))
print(np.shares_memory(arr, second))
second[0] = 999
print(arr)
```

EXPLANATION (BANGLA)

দুই ধাপের basic slicing-এও view বজায় থাকে। তাই second বদলালে original arr-এর সংশ্লিষ্ট position বদলায়. Independent preprocessing দরকার হলে .copy() নাও।

EXPECTED RESULT

দুই shares_memory True; arr-এর index 1 value 999 হয়।

MODULE 17 | NaN, infinity and missing-data handling

12টি problem | N185 থেকে N196

N185 — F-এর missing-value mask

SOLUTION CODE

```
missing_mask = np.isnan(F)
print(missing_mask)
```

EXPLANATION (BANGLA)

np.isnan() প্রতিটি position-এ NaN আছে কি না Boolean হিসেবে জানায়.

EXPECTED RESULT

NaN position-এ True; বাকি position False.

N186 — F-এ মোট ও column-wise missing count

SOLUTION CODE

```
print("Total:", np.isnan(F).sum())
print("By column:", np.isnan(F).sum(axis=0))
```

EXPLANATION (BANGLA)

np.isnan mask-এর True sum করলে missing count পাওয়া যায়. axis=0 column-wise count দেয়.

EXPECTED RESULT

Total=3; প্রতি column-এ [1,1,1].

N187 — Finite value খুঁজে বের করা

SOLUTION CODE

```
values = np.array([1.0, np.nan, np.inf, -np.inf, 5.0])
mask = np.isfinite(values)
print(mask)
print(values[mask])
```

EXPLANATION (BANGLA)

isfinite() NaN ও positive/negative infinity-কে False করে; finite number True.

EXPECTED RESULT

Mask [True,False,False,False,True]; finite values [1.,5.].

N188 — F-এর NaN বাদ দিয়ে overall mean

SOLUTION CODE

```
overall_mean = np.nanmean(F)
print(overall_mean)
```

EXPLANATION (BANGLA)

nanmean NaN ignore করে mean হিসাব করে. তবে F-এর feature column-এর unit আলাদা (যেমন 1.x, 100s); একসঙ্গে pool করা meaningful research statistic নয়.

EXPECTED RESULT

একটি scalar mean print হবে; research interpretation-এ feature unit আলাদা রাখা উচিত.

N189 — F-এর feature-wise mean

SOLUTION CODE

```
means = np.nanmean(F, axis=0)
print(means)
```

EXPLANATION (BANGLA)

axis=0 প্রতিটি feature column আলাদা রাখে; nanmean শুধু ওই column-এর valid value দিয়ে mean করে.

EXPECTED RESULT

[1.825, 0.8, 157.5].

N190 — যে row-তে কোনো NaN নেই তা রাখা

SOLUTION CODE

```
valid_rows = ~np.isnan(F).any(axis=1)
clean = F[valid_rows]
print("Original:", F.shape)
print("Cleaned:", clean.shape)
print(clean)
```

EXPLANATION (BANGLA)

any(axis=1) row-তে অন্তত একটি NaN আছে কি না জানায়. ~ দিয়ে invert করে complete row select হয়.

EXPECTED RESULT

Original shape=(5,3); cleaned shape=(2,3), row 0 ও 4 থাকে.

N191 — Copy-তে NaN-কে 0 দিয়ে replace

SOLUTION CODE

```
clean = F.copy()
clean[np.isnan(clean)] = 0
print(clean)
print("Remaining NaN:", np.isnan(clean).sum())
```

EXPLANATION (BANGLA)

Original F নিরাপদ রাখতে copy নেওয়া হয়েছে. Boolean indexing দিয়ে NaN position-এ 0 assignment.

EXPECTED RESULT

Remaining NaN count=0; F অপরিবর্তিত.

N192 — নিজ নিজ column median দিয়ে NaN fill

SOLUTION CODE

```
clean = F.copy()
medians = np.nanmedian(clean, axis=0)
rows, cols = np.where(np.isnan(clean))
clean[rows, cols] = medians[cols]
print(medians)
print(clean)
```

EXPLANATION (BANGLA)

প্রথমে প্রতিটি column-এর median আলাদা হিসাব. np.where missing row ও column index দেয়; সংশ্লিষ্ট column median বসানো হয়.

EXPECTED RESULT

Column median [1.85,0.8,165]; সব NaN fill হয়.

N193 — Missing বা infinite আছে এমন row mask

SOLUTION CODE

```
test = F.copy()
test[0, 0] = np.inf
bad_rows = ~np.isfinite(test).all(axis=1)
print(bad_rows)
```

EXPLANATION (BANGLA)

isfinite().all(axis=1) row-এর সব value finite কি না পরীক্ষা করে. ~ দিলে অন্তত একটি non-finite value থাকা row চিহ্নিত হয়.

EXPECTED RESULT

Test matrix-এ row 0,1,2,3 bad; row 4 valid.

N194 — Infinity-কে NaN করে nanmean

SOLUTION CODE

```
values = np.array([1.0, np.nan, np.inf, -np.inf, 5.0])
clean = values.copy()
clean[np.isinf(clean)] = np.nan
print(clean)
print(np.nanmean(clean))
```

EXPLANATION (BANGLA)

isinf() positive ও negative infinity শনাক্ত করে. সেগুলো NaN করা হলে nanmean সব non-finite বাদ দিয়ে mean করে.

EXPECTED RESULT

Clean array [1,nan,nan,nan,5]; nanmean=3.0.

N195 — শুধু training-set mean দিয়ে imputation

SOLUTION CODE

```
def impute_from_train(train, other):
    train = np.asarray(train, dtype=float).copy()
    other = np.asarray(other, dtype=float).copy()
    means = np.nanmean(train, axis=0)
    means = np.where(np.isnan(means), 0.0, means)  # all-NaN train column policy
    for data in (train, other):
        r, c = np.where(np.isnan(data))
        data[r, c] = means[c]
    return train, other, means

train_filled, test_filled, train_means = impute_from_train(
    F[:3], F[3:]
)
print(train_means)
print(np.isfinite(train_filled).all(), np.isfinite(test_filled).all())
```

EXPLANATION (BANGLA)

Mean কেবল train data থেকে শেখা হয়; test data-তে সেই একই mean প্রয়োগ হয়. All-NaN training column হলে documented policy হিসেবে 0 নেওয়া হয়েছে.

EXPECTED RESULT

Train ও test উভয় array-তে NaN আর থাকে না; test statistics ব্যবহার হয়নি.

N196 — All-NaN column-এর policy

SOLUTION CODE

```
def impute_all_nan_columns(arr, all_nan_value=0.0):
    x = np.asarray(arr, dtype=float).copy()
    means = np.nanmean(x, axis=0)
    means = np.where(np.isnan(means), all_nan_value, means)
    r, c = np.where(np.isnan(x))
    x[r, c] = means[c]
    return x, means

sample = np.array([[1.0, np.nan], [3.0, np.nan]])
filled, stats = impute_all_nan_columns(sample)
print(stats)
print(filled)
```

EXPLANATION (BANGLA)

যে column পুরো NaN, তার mean নির্ণয় করা যায় না˙ এখানে explicit policy: ওই column-এর missing value 0 দিয়ে fill করা˙ অন্য column-এর mean data থেকে আসে˙

EXPECTED RESULT

Stats [2.,0.]; filled [[1,0],[3,0]].

MODULE 18 | Outliers, scaling and robust summaries

10টি problem | N197 থেকে N206

N197 — Outlier-এর কারণে mean ও median-এর পরিবর্তন

SOLUTION CODE

```
values = np.array([10, 11, 12, 12, 13, 100])
print("Mean:", values.mean())
print("Median:", np.median(values))
```

EXPLANATION (BANGLA)

100 অনেক বড় outlier হওয়ায় mean উপরের দিকে টেনে নেয়. Median sorted middle values-এর ওপর নির্ভর করে' তাই তুলনামূলক robust.

EXPECTED RESULT

Mean≈26.33; median=12.

N198 — T-এর z-score

SOLUTION CODE

```
z = (T - T.mean()) / T.std(ddof=0)
print(z)
```

EXPLANATION (BANGLA)

Z-score=(value-mean)/population standard deviation. প্রতিটি value mean থেকে কত standard deviation দূরে তা দেখায়.

EXPECTED RESULT

Standardized array; mean প্রায় 0 ও population std প্রায় 1.

N199 — A-কে 0-1 min-max scale

SOLUTION CODE

```
scaled = (A - A.min()) / (A.max() - A.min())
print(scaled)
```

EXPLANATION (BANGLA)

Min-max scaling minimum-কে 0 ও maximum-কে 1 করে; মাঝের value proportionalভাবে map হয়.

EXPECTED RESULT

[0.,0.25,0.5,0.75,1.].

N200 — IQR হিসাব

SOLUTION CODE

```
values = np.array([10, 11, 12, 12, 13, 100])
q1, q3 = np.percentile(values, [25, 75])
iqr = q3 - q1
print(q1, q3, iqr)
```

EXPLANATION (BANGLA)

IQR=Q3-Q1; middle 50% data-এর spread বোঝায়.

EXPECTED RESULT

Q1=11.25, Q3=12.75, IQR=1.5.

N201 — IQR rule দিয়ে outlier flag

SOLUTION CODE

```
values = np.array([10, 11, 12, 12, 13, 100])
q1, q3 = np.percentile(values, [25, 75])
iqr = q3 - q1
low, high = q1 - 1.5 * iqr, q3 + 1.5 * iqr
outliers = (values < low) | (values > high)
print(low, high)
print(values[outliers])
```

EXPLANATION (BANGLA)

Conventional fences হলো Q1-1.5*IQR ও Q3+1.5*IQR. Fence-এর বাইরে থাকা value outlier candidate.

EXPECTED RESULT

Lower=9, upper=15; outlier [100].

N202 — 5th ও 95th percentile clipping

SOLUTION CODE

```
values = np.array([1, 2, 3, 4, 5, 100], dtype=float)
lo, hi = np.percentile(values, [5, 95])
clipped = np.clip(values, lo, hi)
print("bounds:", lo, hi)
print("max before/after:", values.max(), clipped.max())
```

EXPLANATION (BANGLA)

Percentile clipping extreme value-কে percentile boundary-তে cap করে. Data row বাদ যায় না.

EXPECTED RESULT

Before max=100; after max=95th percentile boundary.

N203 — X-এর column-wise standardization

SOLUTION CODE

```
mean = X.mean(axis=0)
std = X.std(axis=0)
safe_std = np.where(std == 0, 1, std)
scaled = (X - mean) / safe_std
print(scaled.mean(axis=0))
print(scaled.std(axis=0))
```

EXPLANATION (BANGLA)

প্রতিটি feature আলাদাভাবে center ও scale হয়. Constant column হলে std=0; safe denominator 1 ব্যবহার করা হয়.

EXPECTED RESULT

প্রতিটি column-এর mean প্রায় 0, std প্রায় 1 (constant feature হলে std 0 থাকতে পারে).

N204 — Message length ও URL count আলাদাভাবে scale

SOLUTION CODE

```
length = X[:, 0]
url_count = X[:, 1]
length_scaled = (length - length.min()) / (length.max() - length.min())
url_scaled = (url_count - url_count.min()) / (
    url_count.max() - url_count.min()
)
print(length_scaled)
print(url_scaled)
```

EXPLANATION (BANGLA)

দুটি feature-এর unit ও range আলাদা. তাই প্রতিটি column-এর min ও max আলাদাভাবে ব্যবহার করা হয়েছে; একসঙ্গে combine করা হয়নি.

EXPECTED RESULT

দুটি আলাদা 0-1 scaled vector পাওয়া যায়.

N205 — Median ও IQR দিয়ে robust scaling

SOLUTION CODE

```
med = np.median(X, axis=0)
q1 = np.percentile(X, 25, axis=0)
q3 = np.percentile(X, 75, axis=0)
iqr = q3 - q1
safe_iqr = np.where(iqr == 0, 1, iqr)
robust = (X - med) / safe_iqr
print(robust)
```

EXPLANATION (BANGLA)

Robust scaling-এ mean/std-এর বদলে median/IQR ব্যবহৃত হয়; outlier-এর প্রভাব তুলনামূলক কম. IQR=0 হলে safe denominator 1.

EXPECTED RESULT

প্রতিটি feature median থেকে IQR unit-এ distance পায়.

N206 — Train-only scaling দিয়ে train/test transform

SOLUTION CODE

```
rng = np.random.default_rng(42)
idx = rng.permutation(len(X))
cut = int(0.8 * len(X))
train, test = X[idx[:cut]], X[idx[cut:]]
mean = train.mean(axis=0)
std = train.std(axis=0)
safe_std = np.where(std == 0, 1, std)
train_scaled = (train - mean) / safe_std
test_scaled = (test - mean) / safe_std
print(train_scaled.shape, test_scaled.shape)
```

EXPLANATION (BANGLA)

Split আগে হয়. Mean/std শুধু train থেকে শেখা; একই stats train ও test-এ প্রয়োগ. Test data দিয়ে scaler fit করলে data leakage হয়.

EXPECTED RESULT

৬ sample-এ train shape=(4,3), test shape=(2,3).

MODULE 19 | Vectors, matrices and linear algebra

12টি problem | N207 থেকে N218

N207 — দুই vector-এর dot product

SOLUTION CODE

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print(np.dot(a, b))
```

EXPLANATION (BANGLA)

Dot product corresponding element multiply করে সব যোগ করে: 1*4 + 2*5 + 3*6.

EXPECTED RESULT

32.

N208 — Element-wise multiplication বনাম dot product

SOLUTION CODE

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print(a * b)
print(np.dot(a, b))
```

EXPLANATION (BANGLA)

এখানে * element-wise product vector ফেরত দেয়; dot product scalar sum ফেরত দেয়.

EXPECTED RESULT

Element-wise [4,10,18]; dot product=32.

N209 — 2x3 matrix ও 3x2 matrix multiply

SOLUTION CODE

```
a = np.array([[1, 2, 3], [4, 5, 6]])
b = np.array([[1, 2], [3, 4], [5, 6]])
result = a @ b
print(result)
print(result.shape)
```

EXPLANATION (BANGLA)

Matrix multiplication-এ প্রথম matrix-এর column count দ্বিতীয় matrix-এর row count-এর সমান হতে হয়: 3=3. Output shape বাইরের dimension (2,2).

EXPECTED RESULT

[[22,28],[49,64]], shape=(2,2).

N210 — [3,4] vector-এর Euclidean norm

SOLUTION CODE

```
v = np.array([3, 4])
print(np.linalg.norm(v))
```

EXPLANATION (BANGLA)

Euclidean norm=sqrt(3^2+4^2). Vector-এর length/magnitude পাওয়া যায়.

EXPECTED RESULT

5.0.

N211 — Non-square matrix transpose ও product

SOLUTION CODE

```
m = np.array([[1, 2, 3], [4, 5, 6]])
result = m.T @ m
print(m.T.shape, result.shape)
```

EXPLANATION (BANGLA)

m shape (2,3); transpose shape (3,2). (3,2)@(2,3) করলে output (3,3).

EXPECTED RESULT

Transpose shape=(3,2); product shape=(3,3).

N212 — B-এর সব row-এর pairwise dot product

SOLUTION CODE

```
gram = B @ B.T
print(gram)
print(gram.shape)
```

EXPLANATION (BANGLA)

B @ B.T-তে প্রতিটি row অন্য প্রতিটি row-এর সঙ্গে dot product হয়· একে Gram matrix বলা হয়·

EXPECTED RESULT

shape=(3,3), symmetric matrix.

N213 — Point থেকে [1,1]-এর দূরত্ব

SOLUTION CODE

```
points = np.array([[0, 0], [3, 4], [6, 8]])
target = np.array([1, 1])
distances = np.linalg.norm(points - target, axis=1)
print(distances)
```

EXPLANATION (BANGLA)

Target vector প্রতিটি point থেকে broadcast করে বাদ হয়. norm(axis=1) প্রতিটি row-এর Euclidean distance দেয়·

EXPECTED RESULT

[sqrt(2), sqrt(13), sqrt(74)] ≈ [1.414,3.606,8.602].

N214 — দুই linear equation solve

SOLUTION CODE

```
# 2*x + y = 5; x - y = 1
coeff = np.array([[2, 1], [1, -1]])
rhs = np.array([5, 1])
solution = np.linalg.solve(coeff, rhs)
print(solution)
print(coeff @ solution)
```

EXPLANATION (BANGLA)

Equation system Ax=b আকারে লেখা হয়. linalg.solve() সরাসরি x ও y বের করে; inverse আলাদা করে নেওয়া প্রয়োজন নেই·

EXPECTED RESULT

x=2, y=1; substitution result [5,1].

N215 — 2x2 matrix-এর determinant ও rank

SOLUTION CODE

```
m = np.array([[1, 2], [3, 4]])
print(np.linalg.det(m))
print(np.linalg.matrix_rank(m))
```

EXPLANATION (BANGLA)

Determinant invertibility সম্পর্কে তথ্য দেয়. Rank হলো linearly independent row/column-এর সংখ্যা·

EXPECTED RESULT

Determinant=-2; rank=2.

N216 — Matrix inverse ও identity verification

SOLUTION CODE

```
m = np.array([[2, 1], [1, 1]], dtype=float)
inv = np.linalg.inv(m)
print(inv)
print(np.allclose(m @ inv, np.eye(2)))
```

EXPLANATION (BANGLA)

Inverse A^-1 এমন matrix যাতে A@A^-1=I. Floating-point rounding-এর কারণে equality-র বদলে allclose ব্যবহার করা হয়·

EXPECTED RESULT

Inverse [[1,-1],[-1,2]]; verification True.

N217 — NumPy দিয়ে least-squares coefficient

SOLUTION CODE

```
design = np.array([[1, 0], [1, 1], [1, 2], [1, 3]], dtype=float)
target = np.array([1, 3, 5, 7], dtype=float)
coef, residuals, rank, singular = np.linalg.lstsq(
    design, target, rcond=None
)
print(coef)
```

EXPLANATION (BANGLA)

Design matrix-এর প্রথম column 1 হওয়ায় intercept অন্তর্ভুক্ত. lstsq least-squares error কমায় এমন coefficient বের করে·

EXPECTED RESULT

Coefficient প্রায় [1,2]; অর্থাৎ y=1+2x.

N218 — Linearly dependent column ও rank

SOLUTION CODE

```
m = np.array([[1, 2, 3], [2, 4, 6], [3, 6, 9]], dtype=float)
print("rank:", np.linalg.matrix_rank(m))
print("det:", np.linalg.det(m))
```

EXPLANATION (BANGLA)

দ্বিতীয় ও তৃতীয় column প্রথম column-এর multiple. তাই independent column একটি; rank=1. Matrix singular, inverse নেই।

EXPECTED RESULT

Rank=1; determinant প্রায় 0.

MODULE 20 | Advanced indexing and 3D arrays

10টি problem | N219 থেকে N228

N219 — 2x3x4 3D array তৈরি

SOLUTION CODE

```
arr = np.arange(24).reshape(2, 3, 4)
print(arr.shape, arr.ndim)
```

EXPLANATION (BANGLA)

24টি value-কে 2*3*4=24 dimension-এ সাজানো হয়. ndim axis সংখ্যা দেখায়.

EXPECTED RESULT

shape=(2,3,4), ndim=3.

N220 — 3D array-এর প্রথম 2D slice

SOLUTION CODE

```
arr = np.arange(24).reshape(2, 3, 4)
first_slice = arr[0]
print(first_slice)
print(first_slice.shape)
```

EXPLANATION (BANGLA)

প্রথম axis-এর index 0 নিলে প্রথম 2D block পাওয়া যায়.

EXPECTED RESULT

shape=(3,4); value 0 থেকে 11.

N221 — 3D array-এর index (1,2,3)

SOLUTION CODE

```
arr = np.arange(24).reshape(2, 3, 4)
print(arr[1, 2, 3])
```

EXPLANATION (BANGLA)

3D indexing-এর ক্রম axis 0, axis 1, axis 2. শেষ element-এর coordinate (1,2,3).

EXPECTED RESULT

23.

N222 — 3D array-এর axis-wise sum ও shape

SOLUTION CODE

```
arr = np.arange(24).reshape(2, 3, 4)
for axis in (0, 1, 2):
    result = arr.sum(axis=axis)
    print(axis, result.shape)
```

EXPLANATION (BANGLA)

যে axis বরাবর sum হয়, সেই axis output থেকে বাদ যায়. তাই প্রতিটি axis-এ output shape আলাদা.

EXPECTED RESULT

axis 0: (3,4); axis 1: (2,4); axis 2: (2,3).

N223 — 4x5 RGB image-এর red channel 255

SOLUTION CODE

```
image = np.zeros((4, 5, 3), dtype=np.uint8)
image[:, :, 0] = 255
print(image.shape)
print(image[0, 0])
```

EXPLANATION (BANGLA)

Image shape=(height,width,channels). RGB order-এ channel 0 red; সব pixel-এর red channel 255 করা হয়েছে.

EXPECTED RESULT

shape=(4,5,3); একটি pixel [255,0,0].

N224 — Image-এর channel-wise average

SOLUTION CODE

```
image = np.zeros((4, 5, 3), dtype=np.uint8)
image[:, :, 0] = 255
channel_means = image.mean(axis=(0, 1))
print(channel_means, channel_means.shape)
```

EXPLANATION (BANGLA)

axis=(0,1) height ও width collapse করে; channel axis রেখে প্রতিটি color channel-এর average দেয়.

EXPECTED RESULT

shape=(3,), mean [255,0,0].

N225 — B-এর nonadjacent row 0 ও 2

SOLUTION CODE

```
selected = B[[0, 2]]
print(selected, selected.shape)
```

EXPLANATION (BANGLA)

Integer list indexing/fancy indexing দিয়ে nonadjacent row select করা যায়'

EXPECTED RESULT

[[2,4,6],[14,16,18]], shape=(2,3).

N226 — Coordinate pair দিয়ে advanced indexing

SOLUTION CODE

```
values = B[[0, 2], [1, 0]]
print(values)
```

EXPLANATION (BANGLA)

দুটি index list pairwise coordinate হিসেবে কাজ করে: (row0,col1) এবং (row2,col0).

EXPECTED RESULT

[4 14].

N227 — np.newaxis দিয়ে row ও column vector

SOLUTION CODE

```
v = np.array([1, 2, 3])
column = v[:, np.newaxis]
row = v[np.newaxis, :]
print(column.shape, row.shape)
```

EXPLANATION (BANGLA)

np.newaxis নতুন length-1 axis যোগ করে. Colon-এর আগে/পরে অবস্থান shape নির্ধারণ করে'

EXPECTED RESULT

Column shape=(3,1); row shape=(1,3).

N228 — Image batch-এর তিন channel-এর mean

SOLUTION CODE

```
batch = np.zeros((8, 16, 16, 3))
channel_means = batch.mean(axis=(0, 1, 2))
print(channel_means.shape)
```

EXPLANATION (BANGLA)

Batch, height ও width axis একসঙ্গে reduce করা হয়েছে' শেষ channel axis অক্ষত থাকে'

EXPECTED RESULT

একটি mean per RGB channel; shape=(3,).

MODULE 21 | Saving and loading NumPy research data

10টি problem | N229 থেকে N238

N229 — A-কে .npy save/load ও verify

SOLUTION CODE

```
np.save("A.npy", A)
loaded = np.load("A.npy")
print(loaded.shape)
print(np.array_equal(A, loaded))
```

EXPLANATION (BANGLA)

.npy একটি NumPy array-এর data ও dtype সংরক্ষণ করে. load করার পর shape এবং value equality পরীক্ষা করা হয়েছে.

EXPECTED RESULT

shape=(5,), equality=True.

N230 — X ও y compressed .npz-এ save

SOLUTION CODE

```
np.savez_compressed("research_data.npz", X=X, y=y)
print("Saved X and y")
```

EXPLANATION (BANGLA)

.npz একটি archive-এ একাধিক named array রাখতে পারে. Keyword name-গুলো পরে key হিসেবে ব্যবহার হবে.

EXPECTED RESULT

research_data.npz-এ X ও y সংরক্ষিত.

N231 — .npz file-এর key name দেখা

SOLUTION CODE

```
np.savez_compressed("research_data.npz", X=X, y=y)
with np.load("research_data.npz") as data:
    print(data.files)
    print(data["X"].shape, data["y"].shape)
```

EXPLANATION (BANGLA)

np.load .npz archive খুলে. files property available key list দেয়; key দিয়ে array access হয়.

EXPECTED RESULT

Keys ['X','y']; shapes (6,3) ও (6,).

N232 — B CSV-তে export ও আবার load

SOLUTION CODE

```
np.savetxt("B.csv", B, delimiter=",", fmt="%d")
loaded = np.loadtxt("B.csv", delimiter=",")
print(loaded)
print(loaded.shape)
```

EXPLANATION (BANGLA)

savetxt CSV text file লেখে; delimiter comma. loadtxt একই delimiter দিয়ে numeric matrix পড়ে.

EXPECTED RESULT

Loaded B; shape=(3,3).

N233 — চার decimal precision CSV round-trip

SOLUTION CODE

```
values = np.array([1.234567, 2.345678])
np.savetxt("values.csv", values, fmt="%.4f")
loaded = np.loadtxt("values.csv")
print(loaded)
print(np.allclose(values, loaded, atol=5e-5, rtol=0))
```

EXPLANATION (BANGLA)

%.4f চার decimal place-এ লেখে; ফলে সামান্য rounding error থাকে. atol=0.00005 tolerance দিয়ে approximate equality পরীক্ষা করা হয়েছে.

EXPECTED RESULT

Loaded values চার decimal-এ rounded; allclose সাধারণত True.

N234 — ছোট CSV তৈরি ও (3,2) হিসেবে load

SOLUTION CODE

```
from pathlib import Path
Path("tiny.csv").write_text("1,2\n3,4\n5,6\n")
loaded = np.loadtxt("tiny.csv", delimiter=",", ndmin=2)
print(loaded)
print(loaded.shape)
```

EXPLANATION (BANGLA)

CSV text file-এ তিন row ও দুই column আছে. ndmin=2 এক row হলেও 2D shape বজায় রাখতে সাহায্য করে.

EXPECTED RESULT

shape=(3,2).

N235 — Header row বাদ দিয়ে numeric body load

SOLUTION CODE

```
from pathlib import Path
Path("with_header.csv").write_text("length,url,count\n120,2,4\n45,0,0\n")
loaded = np.loadtxt("with_header.csv", delimiter=",", skiprows=1)
print(loaded)
print(loaded.shape)
```

EXPLANATION (BANGLA)

skiprows=1 প্রথম header line বাদ দেয়; বাকি numeric data load হয়.

EXPECTED RESULT

দুই row, তিন column; shape=(2,3).

N236 — Missing field-সহ CSV genfromtxt

SOLUTION CODE

```
from pathlib import Path
Path("missing.csv").write_text("1,2\n3,\n5,6\n")
loaded = np.genfromtxt("missing.csv", delimiter=",")
print(loaded)
print(np.isnan(loaded))
```

EXPLANATION (BANGLA)

genfromtxt missing numeric field-কে default-এ NaN দিয়ে represent করতে পারে. isnan দিয়ে অবস্থান দেখা হয়.

EXPECTED RESULT

মাঝের row-এর দ্বিতীয় field NaN.

N237 — Temporary directory-তে safe save/load

SOLUTION CODE

```
import tempfile
import os
with tempfile.TemporaryDirectory() as folder:
    path = os.path.join(folder, "A.npy")
    np.save(path, A)
    loaded = np.load(path)
    print(np.array_equal(A, loaded))
```

EXPLANATION (BANGLA)

TemporaryDirectory temporary folder তৈরি করে এবং block শেষ হলে folder মুছে দেয়. Source research file overwrite হওয়ার ঝুঁকি কমে.

EXPECTED RESULT

Equality True; temporary file block শেষে cleanup হয়.

N238 — Feature/label save-load function ও validation

SOLUTION CODE

```
def save_experiment(features, labels, path):
    features = np.asarray(features)
    labels = np.asarray(labels)
    assert features.shape[0] == labels.shape[0]
    np.savez_compressed(path, features=features, labels=labels)

def load_experiment(path):
    with np.load(path) as data:
        features = data["features"]
        labels = data["labels"]
    assert features.shape[0] == labels.shape[0]
    return features, labels

save_experiment(X, y, "experiment.npz")
X2, y2 = load_experiment("experiment.npz")
assert np.array_equal(X, X2) and np.array_equal(y, y2)
print("Data integrity verified")
```

EXPLANATION (BANGLA)

Save করার আগে sample count alignment assert করা হয়. Load করার পরও alignment ও exact value integrity পরীক্ষা করা হয়.

EXPECTED RESULT

Data integrity verified.

MODULE 22 | Vectorization, efficiency and numerical precision

10টি problem | N239 থেকে N248

N239 — এক মিলিয়ন integer-এর sum

SOLUTION CODE

```
arr = np.arange(1, 1_000_001)
print(arr.sum())
```

EXPLANATION (BANGLA)

arange এক মিলিয়ন consecutive integer তৈরি করে. NumPy-এর compiled sum operation পুরো array যোগ করে.

EXPECTED RESULT

500000500000.

N240 — Loop ছাড়া বড় array square

SOLUTION CODE

```
arr = np.arange(1_000_000)
squared = arr ** 2
print(squared[:5], squared.shape)
```

EXPLANATION (BANGLA)

Exponentiation vectorized; প্রতিটি element-এর square হয়, Python for loop দরকার হয় না.

EXPECTED RESULT

প্রথম পাঁচটি [0,1,4,9,16]; shape=(1000000,).

N241 — allclose দিয়ে floating array compare

SOLUTION CODE

```
a = np.array([0.1 + 0.2, 1.0])
b = np.array([0.3, 1.0])
print(a == b)
print(np.allclose(a, b))
```

EXPLANATION (BANGLA)

Floating-point representation-এর কারণে exact equality False হতে পারে. allclose tolerance-এর মধ্যে value কাছাকাছি কি না দেখে.

EXPECTED RESULT

Element-wise equality প্রথম value-তে False; allclose=True.

N242 — 0.1+0.2 exact equality পরীক্ষা

SOLUTION CODE

```
value = 0.1 + 0.2
print(value)
print(value == 0.3)
print(np.isclose(value, 0.3))
```

EXPLANATION (BANGLA)

Binary floating-point-এ 0.1 ও 0.2 exactভাবে represent করা যায় না. তাই result 0.30000000000000004 হতে পারে. isclose tolerance দিয়ে compare করে.

EXPECTED RESULT

Exact equality False; isclose True.

N243 — 100000 element-এ vectorized polynomial

SOLUTION CODE

```
x = np.arange(100_000, dtype=float)
z = x * x + 2 * x + 1
print(z[:5], z.shape)
```

EXPLANATION (BANGLA)

একই formula প্রতিটি x-এ element-wise apply হয়. Python loop ছাড়া NumPy vectorization ব্যবহার করা হয়েছে.

EXPECTED RESULT

প্রথম পাঁচটি [1,4,9,16,25]; shape=(100000,).

N244 — Broadcasting দিয়ে pairwise absolute difference

SOLUTION CODE

```
a = np.array([1, 4, 7])
b = np.array([2, 5])
diff = np.abs(a[:, None] - b[None, :])
print(diff, diff.shape)
```

EXPLANATION (BANGLA)

a[:,None] shape (3,1), b[None,:] shape (1,2). Broadcasting সব pair-এর difference তৈরি করে.

EXPECTED RESULT

[[1,4],[2,1],[5,2]], shape=(3,2).

N245 — NumPy sum ও Python sum fair timing

SOLUTION CODE

```
import timeit
arr = np.arange(1_000_000)
items = arr.tolist()
numpy_times = timeit.repeat(lambda: arr.sum(), repeat=5, number=20)
python_times = timeit.repeat(lambda: sum(items), repeat=5, number=20)
print("NumPy best:", min(numpy_times))
print("Python best:", min(python_times))
```

EXPLANATION (BANGLA)

একই numeric data ও একই repetition count দিয়ে timing করা হয়েছে. repeat-এর একাধিক run থেকে best time তুলনা করা হয়; machine অনুযায়ী ফল বদলাবে.

EXPECTED RESULT

দুইটি elapsed time print হবে; সাধারণত NumPy sum দ্রুত.

N246 — float32 ও float64 memory compare

SOLUTION CODE

```
a = np.ones(1000, dtype=np.float32)
b = np.ones(1000, dtype=np.float64)
print(a.nbytes, b.nbytes)
```

EXPLANATION (BANGLA)

float32 প্রতি value 4 byte; float64 8 byte. একই element count-এ float64 দ্বিগুণ data memory নেয়.

EXPECTED RESULT

4000 ও 8000 bytes.

N247 — Tiny value-এ float32 বনাম float64 precision

SOLUTION CODE

```
a = np.full(1_000_000, 1e-8, dtype=np.float32)
b = np.full(1_000_000, 1e-8, dtype=np.float64)
print(a.sum(dtype=np.float32))
print(b.sum(dtype=np.float64))
```

EXPLANATION (BANGLA)

float32-এর precision কম; অনেক tiny value যোগ করলে rounding error তুলনামূলক বেশি হতে পারে. float64 সাধারণত বেশি precise, তবে বেশি memory নেয়.

EXPECTED RESULT

দুই sum প্রায় 0.01; float64 result সাধারণত বেশি accurate.

N248 — Broadcasting দিয়ে pairwise squared distance

SOLUTION CODE

```
points = np.array([[0., 0.], [3., 4.], [6., 8.]])
diff = points[:, None, :] - points[None, :, :]
squared_distances = np.sum(diff ** 2, axis=2)
print(squared_distances)
print(squared_distances.shape)
```

EXPLANATION (BANGLA)

একটি point-কে অন্য সব point-এর সঙ্গে pair করা হয়. diff shape (N,N,d); শেষ axis-এ squared difference sum করলে (N,N) distance matrix পাওয়া যায়. Memory complexity O(N^2*d).

EXPECTED RESULT

3x3 symmetric matrix; diagonal সব 0.

MODULE 23 | Research feature engineering with NumPy

14টি problem | N249 থেকে N262

N249 — X-এর sample ও feature count

SOLUTION CODE

```
n_samples, n_features = X.shape
print(n_samples, n_features)
```

EXPLANATION (BANGLA)

X-এর row হলো sample এবং column হলো feature. shape থেকে দুইটি count descriptive variable-এ রাখা হয়.

EXPECTED RESULT

6 samples, 3 features.

N250 — X-এর feature column আলাদা variable

SOLUTION CODE

```
message_length = X[:, 0]
url_count = X[:, 1]
suspicious_word_count = X[:, 2]
print(message_length)
print(url_count)
print(suspicious_word_count)
```

EXPLANATION (BANGLA)

Colon সব row নেয়; column index 0,1,2 আলাদা feature vector দেয়.

EXPECTED RESULT

তিনটি 1D vector; প্রতিটির length 6.

N251 — y-তে class 0 ও 1 count

SOLUTION CODE

```
count_0 = np.count_nonzero(y == 0)
count_1 = np.count_nonzero(y == 1)
print(count_0, count_1)
```

EXPLANATION (BANGLA)

Label equality mask দিয়ে class-wise sample count পাওয়া যায়.

EXPECTED RESULT

Class 0=3, class 1=3.

N252 — শুধু label-1 sample-এর feature mean

SOLUTION CODE

```
class1_mean = X[y == 1].mean(axis=0)
print(class1_mean)
```

EXPLANATION (BANGLA)

y==1 দিয়ে scam sample-এর row filter; axis=0 প্রতিটি feature-এর mean দেয়.

EXPECTED RESULT

প্রায় [160,1.667,5.333].

N253 — শুধু label-0 sample-এর feature mean

SOLUTION CODE

```
class0_mean = X[y == 0].mean(axis=0)
print(class0_mean)
```

EXPLANATION (BANGLA)

y==0 দিয়ে non-scam sample select করা হয়; প্রতিটি feature-এর আলাদা mean হিসাব হয়.

EXPECTED RESULT

প্রায় [55,0.333,1.0].

N254 — URL presence-কে fourth feature করা

SOLUTION CODE

```
url_present = (X[:, 1] > 0).astype(int)
X_new = np.column_stack((X, url_present))
print(X_new.shape)
print(X_new[:, -1])
```

EXPLANATION (BANGLA)

URL count থেকে binary feature তৈরি: URL থাকলে 1, না থাকলে 0. column_stack নতুন feature column যোগ করে.

EXPECTED RESULT

shape=(6,4); last column [1,0,1,1,0,0].

N255 — Suspicious-word/message-length ratio

SOLUTION CODE

```
ratio = np.divide(
    X[:, 2], X[:, 0],
    out=np.zeros(X.shape[0], dtype=float),
    where=X[:, 0] != 0
)
print(ratio)
```

EXPLANATION (BANGLA)

প্রতি row-তে suspicious word count / message length. where দিয়ে zero message length-এ division বাদ; output default 0 রাখা হয়েছে·

EXPECTED RESULT

৬টি ratio value; zero length থাকলে তার ratio 0.

N256 — NumPy-only min-max scaling; constant column safe

SOLUTION CODE

```
minimum = X.min(axis=0)
span = X.max(axis=0) - minimum
safe_span = np.where(span == 0, 1, span)
scaled = (X - minimum) / safe_span
print(scaled.min(axis=0), scaled.max(axis=0))
```

EXPLANATION (BANGLA)

Feature-wise min ও range নেওয়া হয়. Constant column হলে denominator 1; সে column-এর scaled values 0 হবে·

EXPECTED RESULT

প্রতিটি nonconstant column min=0, max=1.

N257 — NumPy-only z-score standardization

SOLUTION CODE

```
mean = X.mean(axis=0)
std = X.std(axis=0)
safe_std = np.where(std == 0, 1, std)
z = (X - mean) / safe_std
print(z.mean(axis=0))
print(z.std(axis=0))
```

EXPLANATION (BANGLA)

প্রতিটি feature থেকে নিজস্ব mean বাদ ও std দিয়ে ভাগ. Zero std guard করা হয়েছে·

EXPECTED RESULT

Mean প্রায় 0; nonconstant feature-এর std প্রায় 1.

N258 — Reproducible 80/20 split; X/y aligned

SOLUTION CODE

```
rng = np.random.default_rng(42)
idx = rng.permutation(len(X))
cut = int(0.8 * len(X))
train_idx, test_idx = idx[:cut], idx[cut:]
X_train, X_test = X[train_idx], X[test_idx]
y_train, y_test = y[train_idx], y[test_idx]
print(X_train.shape, X_test.shape)
print(y_train.shape, y_test.shape)
```

EXPLANATION (BANGLA)

একটি shuffled index array তৈরি করে একই index দিয়ে features ও labels select করা হয়েছে· তাই alignment অক্ষত·

EXPECTED RESULT

Train 4 sample, test 2 sample; X shapes (4,3),(2,3).

N259 — Inverse-frequency class weight

SOLUTION CODE

```
classes, counts = np.unique(y, return_counts=True)
n = y.size
k = classes.size
weights = n / (k * counts)
class_weight = dict(zip(classes, weights))
print(class_weight)
```

EXPLANATION (BANGLA)

Formula: weight(class)=N/(K*n_class). Rare class বেশি weight পায়· এই dataset balanced হওয়ায় দুই class-এর weight সমান·

EXPECTED RESULT

{0:1.0, 1:1.0}.

N260 — প্রথম দুই feature-এর correlation matrix

SOLUTION CODE

```
corr = np.corrcoef(X[:, 0], X[:, 1])
print(corr)
print(np.diag(corr))
```

EXPLANATION (BANGLA)

corrcoef দুই feature-এর 2x2 Pearson correlation matrix দেয়. Diagonal-এ প্রতিটি variable-এর নিজের সঙ্গে correlation 1.

EXPECTED RESULT

shape=(2,2); diagonal [1,1].

N261 — Train-fit standardizer ও transform function

SOLUTION CODE

```
def fit_standardizer(train_X):
    mean = train_X.mean(axis=0)
    std = train_X.std(axis=0)
    safe_std = np.where(std == 0, 1, std)
    return {"mean": mean, "std": safe_std}

def transform_standardizer(data, stats):
    return (data - stats["mean"]) / stats["std"]

stats = fit_standardizer(X[:4])
X_train_scaled = transform_standardizer(X[:4], stats)
X_test_scaled = transform_standardizer(X[4:], stats)
print(X_train_scaled.shape, X_test_scaled.shape)
```

EXPLANATION (BANGLA)

fit function শুধু training matrix থেকে mean/std শেখে. transform function একই stats যেকোনো data-তে প্রয়োগ করে; test statistics fit হয় না.

EXPECTED RESULT

Train shape=(4,3), test shape=(2,3).

N262 — 90:10 imbalanced dataset ও balanced subset

SOLUTION CODE

```
rng = np.random.default_rng(42)
X_syn = rng.normal(size=(100, 2))
y_syn = np.array([0] * 90 + [1] * 10)
print(np.unique(y_syn, return_counts=True))

idx0 = rng.choice(np.where(y_syn == 0)[0], 10, replace=False)
idx1 = rng.choice(np.where(y_syn == 1)[0], 10, replace=False)
balanced_idx = rng.permutation(np.concatenate((idx0, idx1)))
X_balanced, y_balanced = X_syn[balanced_idx], y_syn[balanced_idx]
print(np.unique(y_balanced, return_counts=True))
```

EXPLANATION (BANGLA)

প্রথম dataset-এ 90টি class 0 ও 10টি class 1. Minority class count=10; majority থেকেও 10টি random sample নিয়ে balanced subset করা হয়েছে.

EXPECTED RESULT

Original counts [90,10]; balanced subset counts [10,10], total 20 sample.

MODULE 24 | ML math, predictions and evaluation from scratch

12টি problem | N263 থেকে N274

N263 — Correct prediction count

SOLUTION CODE

```
y_true = np.array([1, 0, 1, 1, 0, 0])
y_pred = np.array([1, 0, 0, 1, 1, 0])
correct = np.count_nonzero(y_true == y_pred)
print(correct)
```

EXPLANATION (BANGLA)

একই position-এর true label ও prediction compare হয়. Equality mask-এর True count সঠিক prediction সংখ্যা.

EXPECTED RESULT

4টি correct prediction.

N264 — Accuracy হিসাব

SOLUTION CODE

```
y_true = np.array([1, 0, 1, 1, 0, 0])
y_pred = np.array([1, 0, 0, 1, 1, 0])
accuracy = np.mean(y_true == y_pred)
print(accuracy)
```

EXPLANATION (BANGLA)

Accuracy = correct predictions / total samples. Boolean True=1, False=0; mean নিলে correct fraction পাওয়া যায়.

EXPECTED RESULT

4/6 ≈ 0.6667 বা 66.67%.

N265 — TP, TN, FP, FN count

SOLUTION CODE

```
y_true = np.array([1, 0, 1, 1, 0, 0])
y_pred = np.array([1, 0, 0, 1, 1, 0])
tp = np.sum((y_true == 1) & (y_pred == 1))
tn = np.sum((y_true == 0) & (y_pred == 0))
fp = np.sum((y_true == 0) & (y_pred == 1))
fn = np.sum((y_true == 1) & (y_pred == 0))
print("TP,TN,FP,FN:", tp, tn, fp, fn)
```

EXPLANATION (BANGLA)

Positive class=1 ধরা হয়েছে. TP actual positive ও predicted positive; TN actual negative ও predicted negative; FP false alarm; FN missed positive.

EXPECTED RESULT

TP=2, TN=2, FP=1, FN=1.

N266 — Precision, recall, F1 safe denominator

SOLUTION CODE

```
y_true = np.array([1, 0, 1, 1, 0, 0])
y_pred = np.array([1, 0, 0, 1, 1, 0])
tp = np.sum((y_true == 1) & (y_pred == 1))
fp = np.sum((y_true == 0) & (y_pred == 1))
fn = np.sum((y_true == 1) & (y_pred == 0))
precision = tp / (tp + fp) if tp + fp else 0.0
recall = tp / (tp + fn) if tp + fn else 0.0
f1 = (2 * precision * recall / (precision + recall)
      if precision + recall else 0.0)
print(precision, recall, f1)
```

EXPLANATION (BANGLA)

Precision=TP/(TP+FP); Recall=TP/(TP+FN); F1 দুই metric-এর harmonic mean. Denominator 0 হলে safe policy হিসেবে 0 ফেরত.

EXPECTED RESULT

তিনটিই প্রায় 0.6667.

N267 — Probability থেকে threshold 0.5 prediction

SOLUTION CODE

```
prob = np.array([0.1, 0.7, 0.4, 0.9])
pred = (prob >= 0.5).astype(int)
print(pred)
```

EXPLANATION (BANGLA)

Probability 0.5 বা বেশি হলে class 1; অন্যথায় class 0. >= boundary-কে positive ধরা হয়েছে.

EXPECTED RESULT

[0 1 0 1].

N268 — MSE, MAE ও RMSE

SOLUTION CODE

```
actual = np.array([3.0, 5.0, 7.0])
predicted = np.array([2.5, 5.5, 6.0])
error = actual - predicted
mse = np.mean(error ** 2)
mae = np.mean(np.abs(error))
rmse = np.sqrt(mse)
print(mse, mae, rmse)
```

EXPLANATION (BANGLA)

MSE squared error-এর mean; MAE absolute error-এর mean; RMSE হলো MSE-এর square root, target-এর original unit-এ থাকে.

EXPECTED RESULT

MSE=0.5; MAE≈0.6667; RMSE≈0.7071.

N269 — X @ weights + bias দিয়ে sigmoid

SOLUTION CODE

```
weights = np.array([0.01, 0.2, 0.3])
bias = -1.0
z = X @ weights + bias
probabilities = 1 / (1 + np.exp(-z))
print(z)
print(probabilities)
```

EXPLANATION (BANGLA)

X shape (6,3), weights shape (3,). Matrix-vector product প্রতি sample-এর linear score দেয়; bias যোগ করে sigmoid probability তৈরি হয়.

EXPECTED RESULT

z ও probabilities-এর shape (6,).

N270 — পাঁচ threshold-এ accuracy ও recall

SOLUTION CODE

```
prob = np.array([0.1, 0.7, 0.4, 0.9, 0.6, 0.2])
truth = np.array([0, 1, 1, 1, 0, 0])
thresholds = np.linspace(0.1, 0.9, 5)
pred = prob[:, None] >= thresholds[None, :]
accuracy = np.mean(pred == truth[:, None], axis=0)
tp = np.sum(pred & (truth[:, None] == 1), axis=0)
fn = np.sum((~pred) & (truth[:, None] == 1), axis=0)
recall = np.divide(tp, tp + fn, out=np.zeros(5), where=(tp + fn) != 0)
print(thresholds)
print(accuracy)
print(recall)
```

EXPLANATION (BANGLA)

Probability ও threshold-কে 2D broadcast করে একসঙ্গে পাঁচ threshold-এর prediction হয়. Axis=0-তে metric sample-wise হিসাব হয়.

EXPECTED RESULT

পাঁচটি threshold-এর জন্য accuracy ও recall vector পাওয়া যায়. Threshold বাড়লে সাধারণত positive prediction ও recall কমতে পারে.

N271 — NumPy দিয়ে 2x2 confusion matrix

SOLUTION CODE

```
y_true = np.array([1, 0, 1, 1, 0, 0])
y_pred = np.array([1, 0, 0, 1, 1, 0])
tp = np.sum((y_true == 1) & (y_pred == 1))
tn = np.sum((y_true == 0) & (y_pred == 0))
fp = np.sum((y_true == 0) & (y_pred == 1))
fn = np.sum((y_true == 1) & (y_pred == 0))
cm = np.array([[tn, fp], [fn, tp]])
print(cm)
print(cm.sum(), len(y_true))
```

EXPLANATION (BANGLA)

Convention: row=actual class, column=predicted class; layout [[TN,FP],[FN,TP]]. সব cell যোগ করলে sample count হয়.

EXPECTED RESULT

[[2,1],[1,2]]; sum=6.

N272 — Binary cross-entropy with clipping

SOLUTION CODE

```
y_true = np.array([1, 0, 1, 0])
p = np.array([0.9, 0.2, 0.7, 0.4])
eps = 1e-7
p = np.clip(p, eps, 1 - eps)
loss = -np.mean(
    y_true * np.log(p) + (1 - y_true) * np.log(1 - p)
)
print(loss)
```

EXPLANATION (BANGLA)

Binary cross-entropy positive ও negative label-এর probability penalty করে. clip probability-কে 0 বা 1 থেকে সামান্য দূরে রাখো’ যাতে log(0) না হয়.

EXPECTED RESULT

একটি positive scalar loss পাওয়া যায়; কম হলে predictions সাধারণত ভালো.

N273 — যেকোনো feature count-এর linear prediction

SOLUTION CODE

```
def linear_predict(features, weights, bias):
    features = np.asarray(features)
    weights = np.asarray(weights)
    if features.ndim != 2 or features.shape[1] != weights.size:
        raise ValueError("Feature/weight shape mismatch")
    return features @ weights + bias

weights = np.array([0.1, 0.2, 0.3])
print(linear_predict(X, weights, 1.0))
```

EXPLANATION (BANGLA)

X shape (samples,features), weights shape (features,). Matrix multiplication প্রতিটি row-এর weighted sum করে; bias সব prediction-এ যোগ হয়.

EXPECTED RESULT

একটি prediction per sample; shape=(6,).

N274 — Linear regression-এর এক gradient descent update

SOLUTION CODE

```
features = np.array([[1., 2.], [2., 1.], [3., 2.]])
target = np.array([5., 6., 9.])
weights = np.zeros(2)
bias = 0.0
learning_rate = 0.01

pred = features @ weights + bias
error = pred - target
dw = (2 / len(target)) * (features.T @ error)
db = (2 / len(target)) * error.sum()
weights -= learning_rate * dw
bias -= learning_rate * db
print(weights, bias)
```

EXPLANATION (BANGLA)

প্রথমে prediction ও error. এরপর MSE-এর gradient: dw=(2/n)X.T@error এবং db=(2/n)sum(error). Learning rate দিয়ে parameter update হয়.

EXPECTED RESULT

weights shape=(2,), bias scalar; এক ধাপ update-এর নতুন parameter print হবে.

MODULE 25 | Debugging and output-prediction clinic

14টি problem | N275 থেকে N288

N275 — arange ও linspace output predict

SOLUTION CODE

```
print(np.arange(1, 10, 3))
print(np.linspace(1, 10, 3))
```

EXPLANATION (BANGLA)

arange step 3 ধরে stop 10-এর আগে যায়; linspace 3টি equally spaced value দেয় এবং endpoint রাখে।

EXPECTED RESULT

arange=[1,4,7]; linspace=[1,5.5,10].

N276 — zeros matrix-এ row vector add

SOLUTION CODE

```
result = np.zeros((2, 3)) + np.array([1, 2, 3])
print(result)
print(result.shape)
```

EXPLANATION (BANGLA)

(3,) vector প্রতিটি row-তে broadcast হয়। zeros-এর সঙ্গে যোগ করলে প্রতিটি row vector-এর সমান হয়।

EXPECTED RESULT

[[1,2,3],[1,2,3]], shape=(2,3).

N277 — Different length vector multiply error

SOLUTION CODE

```
a = np.array([1, 2, 3])
b = np.array([4, 5])
try:
    print(a * b)
except ValueError as error:
    print("Shape mismatch:", error)
```

EXPLANATION (BANGLA)

দুই 1D array-এর shape (3,) ও (2,)। কোনো dimension 1 নয় এবং size equal নয়’ তাই element-wise multiplication সম্ভব নয়।

EXPECTED RESULT

ValueError.

N278 — Negative index -2

SOLUTION CODE

```
arr = np.array([1, 2, 3])
print(arr[-2])
```

EXPLANATION (BANGLA)

-1 শেষ element; -2 শেষ থেকে দ্বিতীয় element.

EXPECTED RESULT

2.

N279 — arange slice output

SOLUTION CODE

```
arr = np.arange(10)
print(arr[2:8:2])
```

EXPLANATION (BANGLA)

Start index 2, stop 8 বাদ, step 2; তাই index 2,4,6 নেওয়া হয়।

EXPECTED RESULT

[2 4 6].

N280 — and-এর বদলে & দিয়ে NumPy mask

SOLUTION CODE

```
arr = np.array([1, 3, 5, 7])
mask = (arr > 2) & (arr < 6)
print(arr[mask])
```

EXPLANATION (BANGLA)

Python-এর and scalar Boolean-এর জন্য; NumPy array comparison-এ element-wise & ব্যবহার করতে হয়। প্রতিটি condition bracket-এ রাখতে হবে।

EXPECTED RESULT

[3 5].

N281 — ভুল দৈর্ঘ্যের Boolean mask debug

SOLUTION CODE

```
data = np.arange(18).reshape(6, 3)
mask = np.array([True, False, True, False, True])
try:
    print(data[mask])
except IndexError as error:
    print("Mask error:", error)

# Correct mask: one Boolean per row
correct_mask = np.array([True, False, True, False, True, False])
print(data[correct_mask].shape)
```

EXPLANATION (BANGLA)

Row filter mask-এর length অবশ্যই row count-এর সমান হতে হবে· এখানে data-তে 6 row কিন্তু mask-এ 5 Boolean; তাই error.

EXPECTED RESULT

ভুল mask-এ IndexError; corrected selection shape=(3,3).

N282 — Out-of-range index

SOLUTION CODE

```
arr = np.array([1, 2, 3])
try:
    print(arr[10])
except IndexError as error:
    print(error)
```

EXPLANATION (BANGLA)

Length 3 array-এর valid positive index 0,1,2. Index 10 নেই’ তাই IndexError.

EXPECTED RESULT

IndexError ধরা পড়ে·

N283 — ভুল size reshape

SOLUTION CODE

```
arr = np.array([1, 2, 3])
try:
    print(arr.reshape(2, 2))
except ValueError as error:
    print(error)
```

EXPLANATION (BANGLA)

Original size 3; target shape 2x2-এর জন্য 4টি element দরকার. Element count না মেলায় reshape ব্যর্থ·

EXPECTED RESULT

ValueError.

N284 — astype(int) round করে নাকি truncate

SOLUTION CODE

```
arr = np.array([1.9, -2.9])
print(arr.astype(int))
```

EXPLANATION (BANGLA)

astype(int) fractional part কেটে দেয়; nearest integer-এ round করে না. Negative number-এও zero-এর দিকে truncate হয়·

EXPECTED RESULT

[1 -2].

N285 — Different-unit feature-এ global mean-এর সমস্যা

SOLUTION CODE

```
global_mean = np.nanmean(F)
feature_means = np.nanmean(F, axis=0)
print(global_mean)
print(feature_means)
```

EXPLANATION (BANGLA)

F-এর column-এর scale/unit আলাদা· সব feature একসঙ্গে global mean করলে unit মিশে যায়; feature-wise mean বেশি অর্থবহ·

EXPECTED RESULT

একটি global scalar এবং পৃথক feature means [1.825,0.8,157.5].

N286 — Full dataset scale করার leakage fix

SOLUTION CODE

```
rng = np.random.default_rng(42)
idx = rng.permutation(len(X))
cut = int(0.8 * len(X))
train, test = X[idx[:cut]], X[idx[cut:]]

# Fit statistics on training data only
mean = train.mean(axis=0)
std = train.std(axis=0)
std = np.where(std == 0, 1, std)

train_scaled = (train - mean) / std
test_scaled = (test - mean) / std
print(train_scaled.shape, test_scaled.shape)
```

EXPLANATION (BANGLA)

ভুল পদ্ধতি হলো split-এর আগে full data-তে scaler fit করা। সঠিক পদ্ধতি: আগে split, তারপর train statistics fit, একই stats test-এ apply.

EXPECTED RESULT

Train/test shape বজায় থাকে; test data training statistics-এ transform হয়।

N287 — Floating-point result নিরাপদে compare

SOLUTION CODE

```
a = np.array([[1.0, 2.0], [3.0, 4.0]])
identity = a @ np.linalg.inv(a)
print(identity)
print(np.allclose(identity, np.eye(2)))
```

EXPLANATION (BANGLA)

Numerical linear algebra-তে rounding error হয়; exact == নির্ভরযোগ্য নয়. allclose absolute/relative tolerance দিয়ে approximate equality যাচাই করে।

EXPECTED RESULT

Identity-এর কাছাকাছি matrix; allclose=True.

N288 — Unexpected view mutation fix

SOLUTION CODE

```
original = np.arange(10)
# Risky: view = original[2:6]
safe_view = original[2:6].copy()
safe_view[0] = 999
print(original)
print(safe_view)
```

EXPLANATION (BANGLA)

Slice view original memory share করতে পারে. Independent edit দরকার হলে slice-এর পর .copy() যোগ করাই minimal fix.

EXPECTED RESULT

original অপরিবর্তিত; safe_view-এর প্রথম value 999.

Integration Zone — 10 Research Mini-Projects

এগুলো একাধিক NumPy skill একসঙ্গে ব্যবহার করে. Shared Data Bank-নির্ভর project-গুলো চালানোর আগে Data Bank block run করবে· প্রতিটি project-এর code সম্পূর্ণ program হিসেবে দেওয়া হয়েছে·

C1 — Scam message feature audit | MEDIUM

COMPLETE PROJECT CODE

```
import numpy as np

# X: [message_length, url_count, suspicious_word_count]
# y: 1=scam, 0=non-scam
print("X:", X.shape, X.dtype)
print("y:", y.shape, y.dtype)

classes, counts = np.unique(y, return_counts=True)
print("Class counts:", dict(zip(classes.tolist(), counts.tolist())))

for label in (0, 1):
    class_mean = X[y == label].mean(axis=0)
    print(f"Label {label} feature means:", class_mean)

url_positive_rate = np.mean(X[:, 1] > 0)
url_present = (X[:, 1] > 0).astype(int)
X_extended = np.column_stack((X, url_present))

# Five sanity checks
assert X.ndim == 2 and X.shape[1] == 3
assert y.ndim == 1 and len(y) == len(X)
assert np.isfinite(X).all()
assert set(np.unique(y)).issubset({0, 1})
assert X_extended.shape == (len(X), 4)

print("URL-positive rate:", url_positive_rate)
print("Extended shape:", X_extended.shape)
print("All five sanity checks passed")
```

EXPLANATION (BANGLA)

প্রথমে X ও y-এর shape/dtype দেখা হয়· এরপর class count এবং label অনুযায়ী feature mean বের হয়. URL-positive rate হলো URL count শূন্যের বেশি এমন sample-এর fraction. শেষে URL presence-কে নতুন binary feature হিসেবে যোগ করা হয়· পাঁচটি assert data structure ও label consistency যাচাই করে·

EXPECTED RESULT / INTERPRETATION

Data Bank-এ X shape=(6,3), y shape=(6,); class 0 ও 1-এর count 3 করে; extended X shape=(6,4). এই numeric feature-গুলো message-এর অর্থ/প্রসঙ্গ বোঝে না’ তাই একা scam প্রমাণ করতে পারে না·

C2 — Student exam analysis engine | MEDIUM

COMPLETE PROJECT CODE

```
import numpy as np

def student_averages(scores):
    return scores.mean(axis=1)

def exam_averages(scores):
    return scores.mean(axis=0)

def pass_fail(scores, pass_mark=60):
    # Pass only if every exam reaches the pass mark
    return np.where(np.all(scores >= pass_mark, axis=1),
                    "Pass", "Fail")

def standardize_columns(scores):
    mean = scores.mean(axis=0)
    std = scores.std(axis=0)
    safe_std = np.where(std == 0, 1, std)
    return (scores - mean) / safe_std

original = S.copy()
student_avg = student_averages(S)
exam_avg = exam_averages(S)
best_student = int(np.argmax(student_avg))
labels = pass_fail(S)
standardized = standardize_columns(S)

values, frequencies = np.unique(S, return_counts=True)
print("Student averages:", student_avg)
print("Exam averages:", exam_avg)
print("Top student row:", best_student)
print("Pass/fail:", labels)
print("Score distribution:", dict(zip(values.tolist(), frequencies.tolist())))
print("Standardized shape:", standardized.shape)
assert np.array_equal(S, original)
```

EXPLANATION (BANGLA)

তিনটি reusable function student-wise average, exam-wise average এবং pass/fail নির্ধারণ করে; আরেকটি function column-wise z-score standardization করে. Top student-এর row index argmax দিয়ে পাওয়া যায়. np.unique(..., return_counts=True) score distribution তৈরি করে˙ শেষে assert প্রমাণ করে original S বদলায়নি˙

EXPECTED RESULT / INTERPRETATION

Student averages প্রায় [80.67,70,91.67,59]; exam averages [73,73.25,79.75]; top row index 2; pass/fail [Pass,Pass,Pass,Fail]; standardized shape=(4,3).

C3 — Missing-data laboratory | MEDIUM

COMPLETE PROJECT CODE

```
import numpy as np

missing_mask = np.isnan(F)
missing_rate = missing_mask.mean(axis=0)
print("Missing rate by column:", missing_rate)

# Option 1: drop rows containing any missing value
dropped = F[~missing_mask.any(axis=1)]
print("After drop:", dropped.shape)

# Option 2: median-impute each column
imputed = F.copy()
medians = np.nanmedian(imputed, axis=0)
rows, cols = np.where(np.isnan(imputed))
imputed[rows, cols] = medians[cols]
print("Column medians:", medians)
print("After imputation:", imputed.shape)

# Standardize after imputation
mean = imputed.mean(axis=0)
std = imputed.std(axis=0)
safe_std = np.where(std == 0, 1, std)
standardized = (imputed - mean) / safe_std
assert np.isfinite(standardized).all()

# Policy for a column that is entirely NaN: fill it with 0
def fill_all_nan_columns(data, fallback=0.0):
    data = np.asarray(data, dtype=float).copy()
    col_means = np.nanmean(data, axis=0)
    col_means = np.where(np.isnan(col_means), fallback, col_means)
    r, c = np.where(np.isnan(data))
    data[r, c] = col_means[c]
    return data
```

EXPLANATION (BANGLA)

প্রথমে প্রতি column-এ missing rate বের হয়. Drop strategy-তে যেসব row-তে অন্তত একটি NaN আছে সেগুলো বাদ যায়. Median imputation-এ প্রতিটি column-এর নিজস্ব median বসে. Imputation-এর পরে standardization হয় এবং assert দিয়ে finite value নিশ্চিত করা হয়' পুরো column NaN হলে fallback=0 policy স্পষ্টভাবে নির্ধারণ করা হয়েছে'

EXPECTED RESULT / INTERPRETATION

F-এ প্রতিটি column-এ 1টি করে NaN; drop করলে shape=(2,3), median imputation করলে shape=(5,3); standardized output finite.

C4 — Reproducible ML data split | MEDIUM

COMPLETE PROJECT CODE

```
import numpy as np

def make_experiment(seed=42):
    rng = np.random.default_rng(seed)
    features = rng.normal(size=(200, 5))
    true_weights = np.array([0.8, -0.5, 0.3, 0.2, -0.1])
    scores = features @ true_weights
    labels = (scores > 0).astype(int)

    order = rng.permutation(len(features))
    features, labels = features[order], labels[order]

    cut = int(0.8 * len(features))
    X_train, X_test = features[:cut], features[cut:]
    y_train, y_test = labels[:cut], labels[cut:]

    # Fit normalization on training data only
    mean = X_train.mean(axis=0)
    std = X_train.std(axis=0)
    safe_std = np.where(std == 0, 1, std)
    X_train_scaled = (X_train - mean) / safe_std
    X_test_scaled = (X_test - mean) / safe_std

    return (X_train_scaled, X_test_scaled, y_train, y_test,
            mean, safe_std)

result1 = make_experiment(42)
result2 = make_experiment(42)
assert all(np.array_equal(a, b) for a, b in zip(result1, result2))

np.savez_compressed(
    "reproducible_split.npz",
    X_train=result1[0], X_test=result1[1],
    y_train=result1[2], y_test=result1[3],
    train_mean=result1[4], train_std=result1[5]
)
print(result1[0].shape, result1[1].shape)
print("Repeatable:", True)
```

EXPLANATION (BANGLA)

Function-এর ভেতরে seed 42 দিয়ে random data তৈরি হয়. একই permutation features ও labels-এ প্রয়োগ করে alignment রাখা হয়. 80/20 split-এর পরে mean/std কেবল train set থেকে fit হয়; test-এ একই stats ব্যবহার হয়. একই seed-এ function দুইবার চালিয়ে exact equality assert করা হয়েছে এবং compressed NPZ-এ arrays ও scaler stats save হয়েছে.

EXPECTED RESULT / INTERPRETATION

Train shape=(160,5), test shape=(40,5); দুই run-এর output একই; reproducible_split.npz তৈরি হয়.

C5 — Outlier-robust sensor research | MEDIUM

COMPLETE PROJECT CODE

```
import numpy as np

rng = np.random.default_rng(42)
readings = rng.normal(loc=50, scale=5, size=500)
outlier_idx = rng.choice(500, size=5, replace=False)
readings[outlier_idx] += rng.choice([-1, 1], size=5) * 35

def report(data, label):
    q1, q3 = np.percentile(data, [25, 75])
    iqr = q3 - q1
    low, high = q1 - 1.5 * iqr, q3 + 1.5 * iqr
    flags = (data < low) | (data > high)
    print(label, {
        "mean": data.mean(),
        "median": np.median(data),
        "std": data.std(),
        "IQR": iqr,
        "IQR_outlier_count": int(flags.sum())
    })

report(readings, "Before")
lo, hi = np.percentile(readings, [5, 95])
clipped = np.clip(readings, lo, hi)
report(clipped, "After clipping")
print("Max before/after:", readings.max(), clipped.max())
```

EXPLANATION (BANGLA)

প্রথমে 500টি normal sensor reading তৈরি হয়; পাঁচটি position-এ বড় positive/negative outlier inject করা হয়. Report function mean, median, std, IQR এবং IQR fence-এর বাইরে count দেয়. এরপর 5th ও 95th percentile clipping করে before/after summary তুলনা করা হয়.

EXPECTED RESULT / INTERPRETATION

Outlier-এর কারণে mean/std বেশি প্রভাবিত হবে; median/IQR তুলনামূলক robust. Clipping-এর পরে extreme maximum/minimum সীমিত হয়. Exact values seed-নির্ভর.

C6 — K-nearest-neighbor distance core | HARD

COMPLETE PROJECT CODE

```
import numpy as np

train_X = np.array([
    [0, 0], [0, 1], [1, 0], [1, 1], [2, 1], [2, 0],
    [5, 5], [5, 6], [6, 5], [6, 6], [7, 5], [7, 6]
], dtype=float)
train_y = np.array([0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1])

queries = np.array([[0.5, 0.5], [5.5, 5.5], [1.5, 1.5]])
k = 3

# Shape: (query_count, train_count, coordinate_count)
diff = queries[:, None, :] - train_X[None, :, :]
distances = np.linalg.norm(diff, axis=2)

nearest_idx = np.argsort(distances, axis=1)[:, :k]
nearest_labels = train_y[nearest_idx]
votes = np.eye(2, dtype=int)[nearest_labels].sum(axis=1)
predictions = np.argmax(votes, axis=1)

print("Nearest indices:\n", nearest_idx)
print("Nearest labels:\n", nearest_labels)
print("Predictions:", predictions)
```

EXPLANATION (BANGLA)

Broadcasting দিয়ে প্রতিটি query point থেকে প্রতিটি training point-এর coordinate difference তৈরি হয়. norm(axis=2) সব pair-এর Euclidean distance দেয়. argsort(axis=1) প্রতিটি query-এর nearest k index নেয়. One-hot vote sum করে majority class বেছে নেওয়া হয়; scikit-learn ব্যবহার করা হয়নি.

EXPECTED RESULT / INTERPRETATION

প্রথম query class 0, দ্বিতীয় query class 1 এবং তৃতীয় query-ও class 0-এর কাছাকাছি; prediction vector [0,1,0].

C7 — NumPy-only linear regression | HARD

COMPLETE PROJECT CODE

```
import numpy as np

rng = np.random.default_rng(42)
x = rng.uniform(-5, 5, size=200)
noise = rng.normal(0, 1, size=200)
target = 3 * x + 2 + noise

order = rng.permutation(len(x))
cut = int(0.8 * len(x))
train_idx, test_idx = order[:cut], order[cut:]
x_train, x_test = x[train_idx], x[test_idx]
y_train, y_test = target[train_idx], target[test_idx]

# Design matrix: first column is intercept
X_train = np.column_stack((np.ones(len(x_train)), x_train))
X_test = np.column_stack((np.ones(len(x_test)), x_test))

coef, *_ = np.linalg.lstsq(X_train, y_train, rcond=None)
pred = X_test @ coef

error = y_test - pred
mae = np.mean(np.abs(error))
mse = np.mean(error ** 2)
rmse = np.sqrt(mse)

print("Intercept, slope:", coef)
print("MAE/MSE/RMSE:", mae, mse, rmse)
```

EXPLANATION (BANGLA)

y=3x+2+noise synthetic data তৈরি হয়. Reproducible shuffle দিয়ে train/test split; design matrix-এর প্রথম column intercept. lstsq শুধু train data-তে coefficient fit করে; held-out test-এ prediction ও MAE/MSE/RMSE হিসাব হয়.

EXPECTED RESULT / INTERPRETATION

Estimated intercept প্রায় 2 ও slope প্রায় 3 হবে; noise-এর কারণে exact নয়. Test metrics finite positive values.

C8 — Vectorized image batch inspector | HARD

COMPLETE PROJECT CODE

```
import numpy as np

rng = np.random.default_rng(42)
images = rng.integers(0, 256, size=(10, 32, 32, 3), dtype=np.uint8)

# One mean per image and per RGB channel
per_image_channel_mean = images.mean(axis=(1, 2))  # (10, 3)

# One global mean per RGB channel
global_channel_mean = images.mean(axis=(0, 1, 2))  # (3,)

# One brightness value per image
per_image_brightness = images.mean(axis=(1, 2, 3))  # (10,)

# Count bright pixels per image; a pixel is bright if any channel > 200
bright_pixel_count = (images.max(axis=3) > 200).sum(axis=(1, 2))

print("Image shape:", images.shape)
print("Per-image channel mean:", per_image_channel_mean.shape)
print("Global channel mean:", global_channel_mean.shape)
print("Brightness:", per_image_brightness.shape)
print("Bright-pixel counts:", bright_pixel_count.shape)
```

EXPLANATION (BANGLA)

Image batch-এর shape (batch,height,width,channel). axis=(1,2) spatial dimension collapse করে প্রতি image-এর RGB mean দেয়· axis=(0,1,2) সব image ও pixel collapse করে global RGB mean দেয়. Brightness-এ channel-ও reduce হয়; bright pixel count প্রতি image-এ হিসাব হয়·

EXPECTED RESULT / INTERPRETATION

Shapes: images (10,32,32,3); per-image channel mean (10,3); global channel mean (3,); brightness (10,); bright pixel count (10,).

C9 — Binary classification evaluation lab | HARD

COMPLETE PROJECT CODE

```
import numpy as np

rng = np.random.default_rng(42)
n = 200
y_true = rng.integers(0, 2, size=n)
prob = np.clip(
    0.15 + 0.7 * y_true + rng.normal(0, 0.2, size=n),
    0, 1
)

def evaluate_at_threshold(y_true, prob, threshold):
    y_pred = (prob >= threshold).astype(int)
    tp = np.sum((y_true == 1) & (y_pred == 1))
    tn = np.sum((y_true == 0) & (y_pred == 0))
    fp = np.sum((y_true == 0) & (y_pred == 1))
    fn = np.sum((y_true == 1) & (y_pred == 0))
    accuracy = (tp + tn) / len(y_true)
    precision = tp / (tp + fp) if tp + fp else 0.0
    recall = tp / (tp + fn) if tp + fn else 0.0
    f1 = (2 * precision * recall / (precision + recall)
          if precision + recall else 0.0)
    return (tp, tn, fp, fn, accuracy, precision, recall, f1)

for threshold in (0.3, 0.5, 0.7):
    print(threshold, evaluate_at_threshold(y_true, prob, threshold))
```

EXPLANATION (BANGLA)

Seed দিয়ে 200টি true label ও probability তৈরি হয়. evaluate function threshold থেকে prediction বানায়, TP/TN/FP/FN গোনে এবং accuracy, precision, recall, F1 হিসাব করে. Threshold বদলে metric কীভাবে বদলায় তা দেখা যায়·

EXPECTED RESULT / INTERPRETATION

প্রতিটি threshold-এর জন্য confusion counts ও চারটি metric print হবে. Seed 42 একই run পুনরুৎপাদন করে·

C10 — Research-ready NumPy preprocessing pipeline | HARD

COMPLETE PROJECT CODE

```
import numpy as np
import json

rng = np.random.default_rng(42)
raw_X = rng.normal(size=(120, 4))
raw_X[:, 3] = 5.0  # constant feature
raw_y = rng.integers(0, 2, size=120)

# Insert missing values in numeric features
missing = rng.random(raw_X.shape) < 0.05
raw_X[missing] = np.nan

order = rng.permutation(len(raw_X))
cut = int(0.8 * len(raw_X))
train_idx, test_idx = order[:cut], order[cut:]
train_X, test_X = raw_X[train_idx].copy(), raw_X[test_idx].copy()
train_y, test_y = raw_y[train_idx], raw_y[test_idx]

# Learn imputation values from training data only
impute_values = np.nanmean(train_X, axis=0)
impute_values = np.where(np.isnan(impute_values), 0.0, impute_values)

def apply_imputation(data, values):
    data = data.copy()
    r, c = np.where(np.isnan(data))
    data[r, c] = values[c]
    return data

train_filled = apply_imputation(train_X, impute_values)
test_filled = apply_imputation(test_X, impute_values)

# Learn scaling parameters from imputed training data only
mean = train_filled.mean(axis=0)
std = train_filled.std(axis=0)
safe_std = np.where(std == 0, 1.0, std)

train_scaled = (train_filled - mean) / safe_std
test_scaled = (test_filled - mean) / safe_std

assert np.isfinite(train_scaled).all()
assert np.isfinite(test_scaled).all()
assert len(train_scaled) == len(train_y)
assert len(test_scaled) == len(test_y)

metadata = {
    "seed": 42,
    "imputation": "training-column mean; all-NaN column -> 0",
    "scaling": "training mean and population std",
    "zero_std_policy": "use denominator 1"
}
np.savez_compressed(
    "preprocessed_research_data.npz",
    X_train=train_scaled, X_test=test_scaled,
    y_train=train_y, y_test=test_y,
    impute_values=impute_values, mean=mean, std=safe_std,
    metadata=json.dumps(metadata)
)
print(train_scaled.shape, test_scaled.shape)
print("All finite and aligned")
```

EXPLANATION (BANGLA)

প্রথমে missing value ও constant column-সহ synthetic data তৈরি হয়. Split করার পর imputation mean কেবল train থেকে শেখা হয়; all-NaN feature হলে 0 policy. Filled train data থেকে scaling mean/std শেখা হয়; একই stats test-এ প্রয়োগ. Finite value ও X/y alignment assert করা হয়. NPZ-এ arrays, learned preprocessing stats এবং metadata save হয়.

EXPECTED RESULT / INTERPRETATION

Train shape=(96,4), test shape=(24,4); সব transformed value finite; label alignment ঠিক; preprocessed_research_data.npz তৈরি হয়.

Final Revision Checklist

নিজেকে NumPy-তে confident মনে করার আগে নিচের বিষয়গুলো tutorial ছাড়া করতে পারো কি না যাচাই করো:

[ ] Array create, shape/dtype/ndim inspect এবং casting করতে পারি.

[ ] 1D/2D/3D indexing, slicing, view ও copy-এর পার্থক্য বুঝি.

[ ] reshape, transpose, stack, split করে shape track করতে পারি.

[ ] Element-wise arithmetic ও matrix multiplication আলাদা করতে পারি.

[ ] axis=0, axis=1, keepdims ও broadcasting explain করতে পারি.

[ ] Boolean mask দিয়ে loop ছাড়া row filter করতে পারি.

[ ] Mean, median, variance, standard deviation, percentile ও correlation হিসাব করতে পারি.

[ ] Seed দিয়ে reproducible random data তৈরি এবং X/y alignment বজায় রাখতে পারি.

[ ] NaN, infinity, outlier, zero variance ও feature-wise scaling handle করতে পারি.

[ ] Train-only preprocessing, ML prediction ও evaluation metric NumPy দিয়ে implement করতে পারি.

What to Study Next

এই workbook শেষ করার পর Pandas দিয়ে real CSV data cleaning, Matplotlib দিয়ে research visualization, scikit-learn দিয়ে model training এবং thesis-এর প্রয়োজন অনুযায়ী text/image processing শেখা ভালো. NumPy-এর shape, masking, statistics ও preprocessing পরিষ্কার থাকলে পরের ধাপগুলো অনেক সহজ হবে.

এই guide-এর সব 288টি problem ও 10টি project-এর code block syntax-checked; প্রতিটি snippet shared data bank-সহ isolated environment-এ run করে যাচাই করা হয়েছে. Randomized output seed-নির্ভর এবং timing result machine-নির্ভর.